CSE401

Artificial

Intelligence

Practical File

Submitted to

AMITY UNIVERSITY, UTTAR PRADESH

In partial fulfilment of the requirements for the award of the degree ofBachelor of Technology

In

Computer Science & Engineering

Submitted By:

Pothina Venkata Sai Jaswanth Kumar

Submitted to:

Dr. Nidhi Mishra

DEPARTMENT OF AMITY SCHOOL OF ENGINEERING

AMITY UNIVERSITY UTTAR PRADESH

TABLE OF CONTENTS

S. No	PROGRAM	DATE	Page No	Signature
1	Introduction to Python Design an And, OR and XOR gate and its truth table using python.	05-01-23		
2	Write a program to implement BFS for water jug problem using Python.	12-01-23		
3	Write a program to implement DFS using python.	19-01-23		
4	Write a program to solve an 8-puzzle problem using A* algorithm in python.	26-01-23		
5	To implement 8 Puzzle Single Player Game using Breadth First Search.	02-01-23		
6	To implement Tic-Tac-Toe game using MiniMax Algorithm. To implement Tic-Tac-Toe game using Alpha Beta pruning Algorithm.	16-02-23 23-02-23		
7	Write a python program for the cryptarithmetic problem.	02-03-23		
8	To implement graph colouring problem in python.	23-03-23		
9	Write a program for tokenization of word and sentence using NLTK package in python. Also perform: . Stop word removal . Stemming . Lemmatization . POS tagging (parsing) . Parsing of a sentence	12-04-23		
10	To implement 0/1 Knapsack using Brute Force approach	12-04-23		
11	To study about Fuzzy Scikit Libraries	12-04-23		
12	OPEN ENDED-EXPERIMENT (Fake News Detection)	12-04-23		

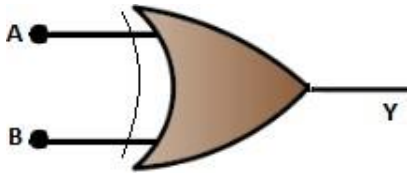
LAB-1

Aim: Write a program to print truth table of XOR, AND, OR.

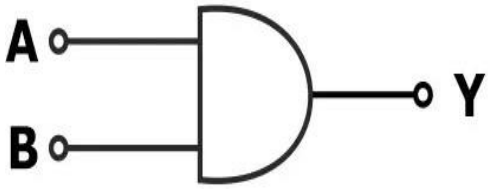
Language: Python

Theory:

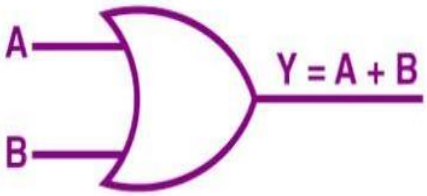
Exclusive-OR gate or XOR gate is achieved by combining standard logic gates together. XOR gate is used extensively in error detection circuits, computational logic comparators and arithmetic logic circuits. The Exclusive OR gate gives an output only if its two inputs are dissimilar, namely if one of them is high (one) and the other is low (zero).



AND gate is one of the digital system's primary or fundamental logic gates. A logic gate is a physical device that performs a logical operation in one or more logical fain-in orinputs and produces a logical output. The output of a "AND Gate" only returns "HIGH" when all the inputs are "HIGH".



OR gate may have two or more inputs as per the need of the application, but it has only one output just like AND gate. In this logic gate, the output is logic 1 when at least one of its inputs is set to 1 or HIGH. In this logic gate output is logic 0 when all the inputs are set to 0 orLOW.



Source Code:

```
print("Implementation of gates using python ")
print("\nselect 1 for AND gate \nselect 2 for OR gate \nselect 3 for XOR gate")

def XOR(a,b):
  if(a!=b):
    return 1
  else:
    return 0
def AND(a,b):
  if(a==1 and b==1):
    return 1
  else:
    return 0
def OR(a,b):
```

```
  if(a==1 or b==1):
    return 1
  else:
    return 0
a = int(input("\nenter the first number"))
b = int(input("\nenter the second number"))
c = int(input("\n enter the choice"))
if(c==1):
  print("\n Your result is:",AND(a,b))
elif(c==2):
  print("\n Your result is: ",OR(a,b))
elif(c==3):
  print("\n Your result is: ",XOR(a,b))
else:
  print("Worng Choice")
print("\n printing the table of the given gates:")
print("|-------------------------|")
print("|       XOR GATE          |")
print("|-------------------------|")
print("|   A   |   B   |    X    |")
print("|_________________________|")
print("|   0   |   0   |","  ",XOR(0,0) ,"   |")
print("|_________________________|")
print("|   0   |   1   |","  ",XOR(0,1) ,"   |")
print("|_________________________|")
print("|   1   |   0   |","  ",XOR(1,0) ,"   |")
print("|_________________________|")
print("|   1   |   1   |","  ",XOR(1,1) ,"   |")
print("|_________________________|")
print("|-------------------------|")
print("|       OR GATE           |")
print("|-------------------------|")
print("|   A   |   B   |    X    |")
print("|_________________________|")
print("|   0   |   0   |","  ",OR(0,0) ,"   |")
print("|_________________________|")
print("|   0   |   1   |","  ",OR(0,1) ,"   |")
print("|_________________________|")
print("|   1   |   0   |","  ",OR(1,0) ,"   |")
print("|_________________________|")
print("|   1   |   1   |","  ",OR(1,1) ,"   |")
print("|_________________________|")
print("|-------------------------|")
print("|       AND GATE          |")
print("|-------------------------|")
print("|   A   |   B   |    X    |")
print("|_________________________|")
print("|   0   |   0   |","  ",AND(0,0) ,"   |")
print("|_________________________|")
print("|   0   |   1   |","  ",AND(0,1) ,"   |")
print("|_________________________|")
print("|   1   |   0   |","  ",AND(1,0) ,"   |")
print("|_________________________|")
print("|   1   |   1   |","  ",AND(1,1) ,"   |")
print("|_________________________|")
```

Output:

```
Implementation of gates using python

select 1 for AND gate
select 2 for OR gate
select 3 for XOR gate

enter the first number1

enter the second number1

 enter the choice1

 Your result is: 1

 printing the table of the given gates:
|------------------------|
|        XOR GATE        |
|------------------------|
|   A   |   B   |   X    |
|________________________|
|   0   |   0   |   0    |
|________________________|
|   0   |   1   |   1    |
|________________________|
|   1   |   0   |   1    |
|________________________|
|   1   |   1   |   0    |
|________________________|
|------------------------|
|        OR GATE         |
|------------------------|
|   A   |   B   |   X    |
|________________________|
|   0   |   0   |   0    |
|________________________|
|   0   |   1   |   1    |
|________________________|
|   1   |   0   |   1    |
|________________________|
|   1   |   1   |   1    |
|________________________|
|------------------------|
|        AND GATE        |
|------------------------|
|   A   |   B   |   X    |
|________________________|
|   0   |   0   |   0    |
|________________________|
|   0   |   1   |   0    |
|________________________|
|   1   |   0   |   0    |
|________________________|
|   1   |   1   |   1    |
|________________________|
```

Sets:
Sets are used to store multiple items in a single variable.
Set is one of 4 built-in data types in Python used to store collections of data, the other 3 are List, Tuple, and Dictionary, all with different qualities and usage.
A set is a collection which is unordered, unchangeable*, and unindexed.

Source code:

```
print("Set implementations in python: ")
myset=set(["a","b","c"])
print(myset)
print("\nAdding elements to the set")
myset.add("d")
print(myset)
print("\n Updating element to the set")
myset.update(["d","g"])
print(myset)
print('\nDiscarding element from the set')
myset.discard("d")
print(myset)
```

Output:

```
Set implementations in python:
{'c', 'b', 'a'}

Adding elements to the set
{'c', 'd', 'b', 'a'}

 Updating element to the set
{'g', 'c', 'd', 'a', 'b'}

Discarding element from the set
{'g', 'c', 'a', 'b'}
```

LAB-2

Aim: Write a program in python to implement BFS water jug problem in python.

Theory: You are given an m liter jug and a n liter jug. Both the jugs are initially empty. The jugs don't have markings to allow measuring smaller quantities. You have to use the jugs to measure d liters of water where d is less than n.
(X, Y) corresponds to a state where X refers to the amount of water in Jug1 and Y refers to theamount of water in Jug2
Determine the path from the initial state (xi, yi) to the final state (xf, yf), where (xi, yi) is (0, 0) which indicates both Jugs are initially empty and (xf, yf) indicates a state which could be (0, d) or(d, 0).
The operations you can perform are:

Empty a Jug, (X, Y)->(0, Y) Empty Jug 1Fill a Jug,
(0, 0)->(X, 0) Fill Jug 1
Pour water from one jug to the other until one of the jugs is either empty or full, (X, Y) ->(X-d, Y+d)

Source Code:

```
from collections import deque

def BFS(a, b, target):

    m = {}
    isSolvable = False
    path = []

    # Queue to maintain states
    q = deque()

    # Initializing with initial state
    q.append((0, 0))

    while (len(q) > 0):

        # Current state
        u = q.popleft()

        if ((u[0], u[1]) in m):
            continue
        # Doesn't met jug constraints
        if ((u[0] > a or u[1] > b or
             u[0] < 0 or u[1] < 0)):
            continue
        # Filling the vector for constructing
        # the solution path
        path.append([u[0], u[1]])
        # Marking current state as visited
        m[(u[0], u[1])] = 1

        # If we reach solution state, put ans=1
        if (u[0] == target or u[1] == target):
            isSolvable = True
            if (u[0] == target):
```

```
            if (u[1] != 0):
                # Fill final state
                path.append([u[0], 0])
        else:
            if (u[0] != 0):
                # Fill final state
                path.append([0, u[1]])
        # Print the solution path
        sz = len(path)
        for i in range(sz):
            print("(", path[i][0], ",",
                  path[i][1], ")")
        break
    q.append([u[0], b])
    q.append([a, u[1]])
    for ap in range(max(a, b) + 1):
        # Pour amount ap from Jug2 to Jug1
        c = u[0] + ap
        d = u[1] - ap
        # Check if this state is possible or not
        if (c == a or (d == 0 and d >= 0)):
            q.append([c, d])
        # Pour amount ap from Jug 1 to Jug2
        c = u[0] - ap
        d = u[1] + ap
        # Check if this state is possible or not
        if ((c == 0 and c >= 0) or d == b):
            q.append([c, d])
    q.append([a, 0])
    q.append([0, b])
# No, solution exists if ans=0
if (not isSolvable):
    print("No solution")
if __name__ == '__main__':

    Jug1, Jug2, target = 4, 3, 2
    print("Path from initial state "
          "to solution state ::")

    BFS(Jug1, Jug2, target)
```

Output:

```
Path from initial state to solution state ::
( 0 , 0 )
( 0 , 3 )
( 4 , 0 )
( 4 , 3 )
( 3 , 0 )
( 1 , 3 )
( 3 , 3 )
( 4 , 2 )
( 0 , 2 )
```

LAB- 3

Aim- Write a program to implement DFS using python.

Theory

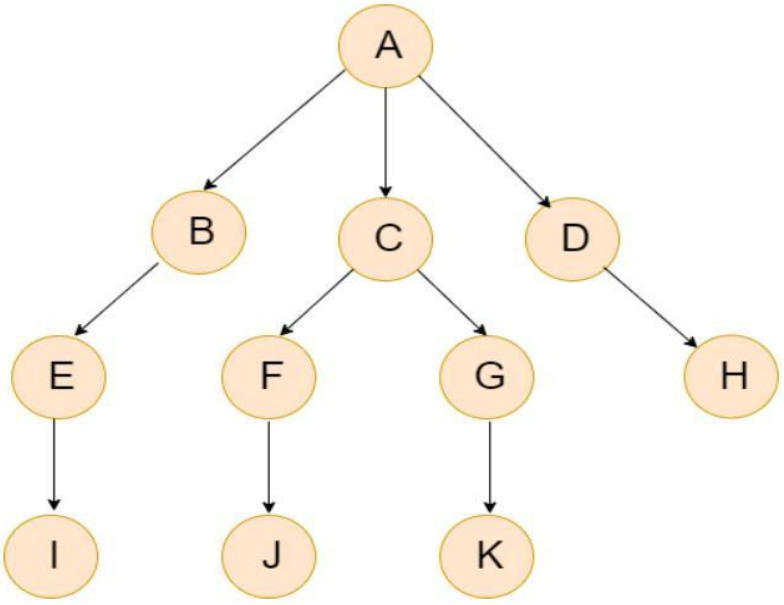
Depth-first search (DFS) is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking. Extra memory, usually a stack, is needed to keep track of the nodes discovered so far along a specified branch which helps in backtracking of the graph.

The DFS algorithm:

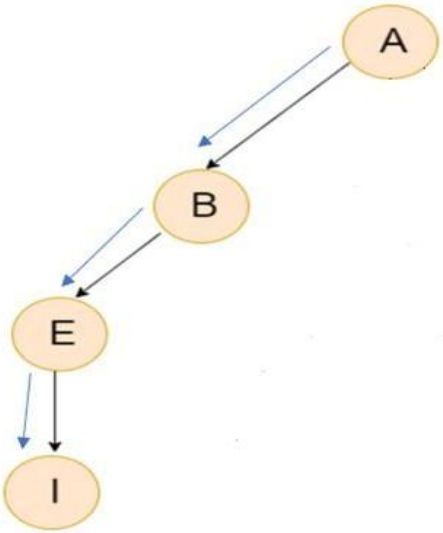
1. Start by putting any one of the graph's vertices on top of a stack.
2. Take the top item of the stack and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to thetop of the stack.
4. Keep repeating steps 2 and 3 until the stack is empty.

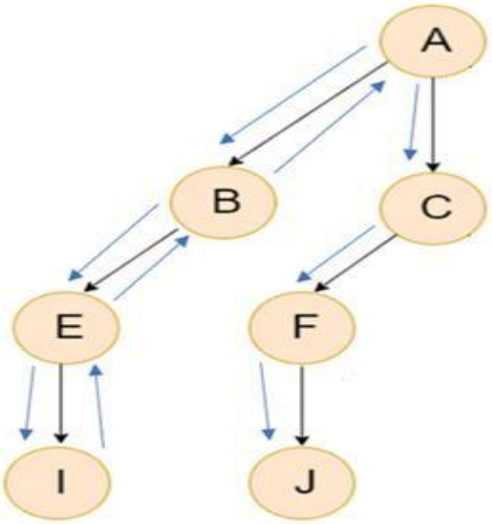
```
DFS(visited, u):
    visited[u]=True
    Print u
    for each v in adj[u]:
        if(visited[v]=False)
            DFS(visited,v)
```

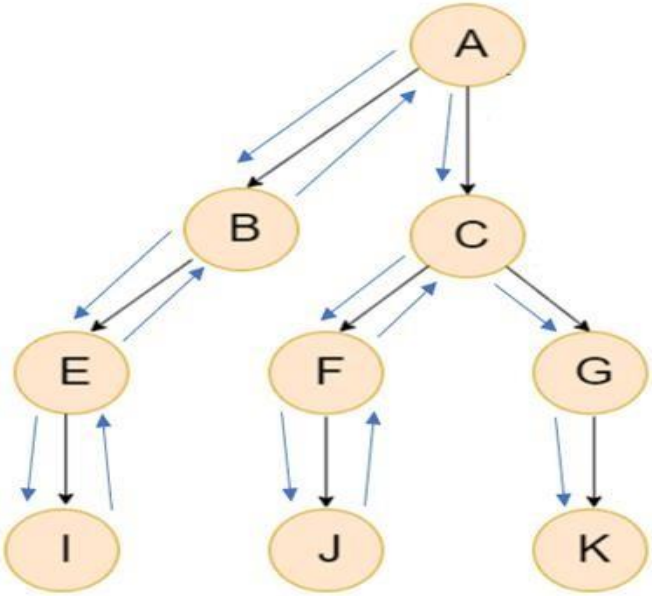
O (V + E)

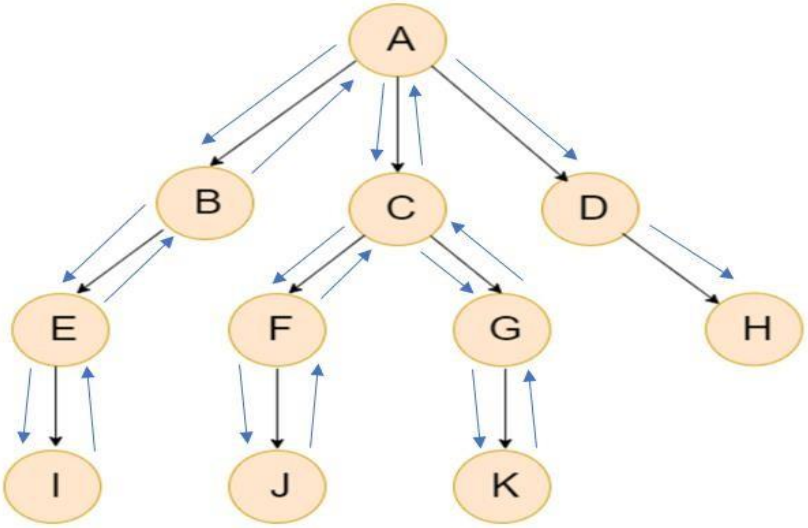


Graph used for the program.


A
B
E
I


A
B
C
E
F
I
J


A
B
C
E
F
G
I
J
K


A
B
C
D
E
F
G
H
I
J
K

Source code:

```
def dfs_recursive(graph, source,path = []):

    if source not in path:
        path.append(source)

        if source not in graph:
            # leaf node, backtrack
            return path

        for neighbour in graph[source]:

            path = dfs_recursive(graph, neighbour, path)


    return path

graph = {"A":["B","C","D"],
  "B":["E"],
  "C":["G","F"],
  "D":["H"],
  "E":["I"],
  "F":["J"],
  "G":["K"]}
dfs_element = dfs_recursive(graph, 'A')
print(dfs_element)
```

Output:

```
Following is the Depth-First Path:
['A', 'B', 'E', 'I', 'C', 'G', 'K', 'F', 'J', 'D', 'H']
```

Lab 4

Aim- Write a program to solve an 8 puzzle problem using A* algorithm in python.

Theory

A* Search Algorithm is a simple and efficient search algorithm that can be used to find the optimal path between two nodes in a graph. It will be used for the shortest path finding. It is an extension of Dijkstra's shortest path algorithm (Dijkstra's Algorithm). The extension here is that, instead of using a priority queue to store all the elements, we use heaps (binary trees) to store them. The A* Search Algorithm also uses a heuristic function that provides additional information regarding how far away from the goal node we are. This function is used in conjunction with the f-heap data structure in order to make searching more efficient.

h(n) - It is the heuristic value of a node. In this program h(n) is the number of displaced tiles (not counting the spaces).

g(n) - It is the distance travelled from one node to another. In this program it is the depth of the node.

f(n) - It denotes the cost of travelling from one node to another.

$$f(n) = g(n) + h(n)$$

A* Algorithm	Greedy Algorithm
A* uses a heuristics to find the solution as quickly as possible.	Greedy algorithms don't use a heuristic value and simply make the locally optimal choice at each step.
A* uses both the cost to reach a node and a heuristic that estimates the cost to get from that node to the goal. This allows A* to prioritize exploring paths that are likely to lead to the goal, resulting in faster convergence.	Greedy only use the cost and choose the path with the lowest estimated cost to the goal, without considering the actual cost to reach that node. This can result in the algorithm getting stuck in suboptimal paths.
A* is more efficient than greedy algorithms as it uses both the cost to reach a node and a heuristic.	Greedy algorithms are less efficient than A* since they only consider cost of reaching a node.

2	8	3
1	6	4
7		5

Initial State

1	2	3
8		4
7	6	5

Final State

Code

```
class Node:
    def __init__(self,data,level,fval):
        """ Initialize the node with the data, level of the node and the calculated fvalue """
        self.data = data
        self.level = level
        self.fval = fval

    def generate_child(self):
        """ Generate child nodes from the given node by moving the blank space
            either in the four directions {up,down,left,right} """
        x,y = self.find(self.data,'_')
        """ val_list contains position values for moving the blank space in either of
            the 4 directions [up,down,left,right] respectively. """
        val_list = [[x,y-1],[x,y+1],[x-1,y],[x+1,y]]
        children = []
        for i in val_list:
            child = self.shuffle(self.data,x,y,i[0],i[1])
            if child is not None:
                child_node = Node(child,self.level+1,0)
                children.append(child_node)
        return children

    def shuffle(self,puz,x1,y1,x2,y2):
        """ Move the blank space in the given direction and if the position value are out
            of limits the return None """
        if x2 >= 0 and x2 < len(self.data) and y2 >= 0 and y2 < len(self.data):
            temp_puz = []
            temp_puz = self.copy(puz)
            temp = temp_puz[x2][y2]
            temp_puz[x2][y2] = temp_puz[x1][y1]
            temp_puz[x1][y1] = temp
            return temp_puz
        else:
            return None


    def copy(self,root):
        """ Copy function to create a similar matrix of the given node"""
        temp = []
        for i in root:
            t = []
            for j in i:
```

```
            t.append(j)
        temp.append(t)
    return temp

    def find(self,puz,x):
        """ Specifically used to find the position of the blank space """
        for i in range(0,len(self.data)):
            for j in range(0,len(self.data)):
                if puz[i][j] == x:
                    return i,j


class Puzzle:
    def __init__(self,size):
        """ Initialize the puzzle size by the specified size,open and closed lists to empty """
        self.n = size
        self.open = []
        self.closed = []

    def accept(self):
        """ Accepts the puzzle from the user """
        puz = []
        for i in range(0,self.n):
            temp = input().split(" ")
            puz.append(temp)
        return puz

    def f(self,start,goal):
        """ Heuristic Function to calculate hueristic value f(x) = h(x) + g(x) """
        return self.h(start.data,goal)+start.level

    def h(self,start,goal):
        """ Calculates the different between the given puzzles """
        temp = 0
        for i in range(0,self.n):
            for j in range(0,self.n):
                if start[i][j] != goal[i][j] and start[i][j] != '_':
                    temp += 1
        return temp


    def process(self):
        """ Accept Start and Goal Puzzle state"""
        print("Enter the start state matrix \n")
```

```
        start = self.accept()
        print("\nEnter the goal state matrix \n")
        goal = self.accept()

        start = Node(start,0,0)
        start.fval = self.f(start,goal)
        """ Put the start node in the open list"""
        self.open.append(start)
        print("\n")
        while True:
            cur = self.open[0]
            print("")
            print("  | ")
            print("  | ")
            print(" \\\'/ \n")
            for i in cur.data:
                for j in i:
                    print(j,end=" ")
                print("")
            """ If the difference between current and goal node is 0 we have reached the goal node"""
            if(self.h(cur.data,goal) == 0):
                break
            for i in cur.generate_child():
                i.fval = self.f(i,goal)
                self.open.append(i)
            self.closed.append(cur)
            del self.open[0]

            """ sort the opne list based on f value """
            self.open.sort(key = lambda x:x.fval,reverse=False)


puz = Puzzle(3)
puz.process()
```

Output

IDLE Shell 3.11.1

File Edit Shell Debug Options Window Help

```
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec  6 2022, 19:58:39) [MSC v
Type "help", "copyright", "credits" or "license()" for more informa
>>>
============= RESTART: C:\Users\sumit\Desktop\Sem 6\AI\py\Lab 4.py
Enter the start state matrix

2 8 3
1 6 4
7 _ 5

Enter the goal state matrix

1 2 3
8 _ 4
7 6 5



  |
  |
 \'/

2 8 3
1 6 4
7 _ 5

  |
  |
 \'/

2 8 3
1 _ 4
7 6 5

  |
  |
 \'/

2 8 3
_ 1 4
7 6 5

  |
  |
 \'/

2 _ 3
1 8 4
7 6 5

  |
  |
 \'/

_ 2 3
1 8 4
7 6 5

  |
  |
 \'/

1 2 3
_ 8 4
7 6 5

  |
  |
 \'/

1 2 3
8 _ 4
7 6 5
```

Lab 5

Aim- To implement 8 Puzzle Single Player Game using Breadth First Search.

Introduction

An instance of the n-puzzle game consists of a board holding n^2 -1 distinct movable tiles, plus an empty space. The tiles are numbers from the set 1….. n^2 -1. For any such board the empty space may be legally swapped with any tile horizontally or vertically adjacent to it. In this assignment the blank space is going to be represented with the number 0. Given an initial state fo the board, the combinatorial search problem is to find a sequence of moves that transitions this state to the goal state that is the configuration with all tiles arranged in ascending order 0, 1, … n^2 -1.

The search space is the set of all possible states from reachable from the initial state. The blank space may be swapped with a component in one of the four directions {'Up', 'Down', 'Left', 'Right'}, one move at a time.

In this 8 puzzle problem a 3 into 3 board with 8 tiles (every tile has one number from 1 to 8) and one empty space. The objective is to place the number on tiles to match final configuration using the empty space. We can slide four adjacent (left, right, above and below) tiles into the empty space.

Breadth First Search (BFS)

We can perform a breadth first search on state space (Set of all configurations of a given problem i.e. all states that can be reached from the initial state) tree

Algorithm Review

The searches begin by visiting the root node of the search tree, given by the initial state. Among other book-keeping details, three major things happen in sequence in order to visit a node:

1. First, we remove a node from the frontier set.
2. Second, we check the state against the goal state to determine if a solution has been found.
3. Finally, if the result of the check is negative, we then expand the node. To expand a given node, we generate successor nodes adjacent to the current node, and add them to the frontier set. Note that if these successor nodes are already in the frontier, or have already been visited, then they should not be added to the frontier again.

Initial state:

1	2	5
3	4	
6	7	8

The nodes expanded by BFS (also the nodes that are in the fringe / frontier of the queue) are shown in the following figure:

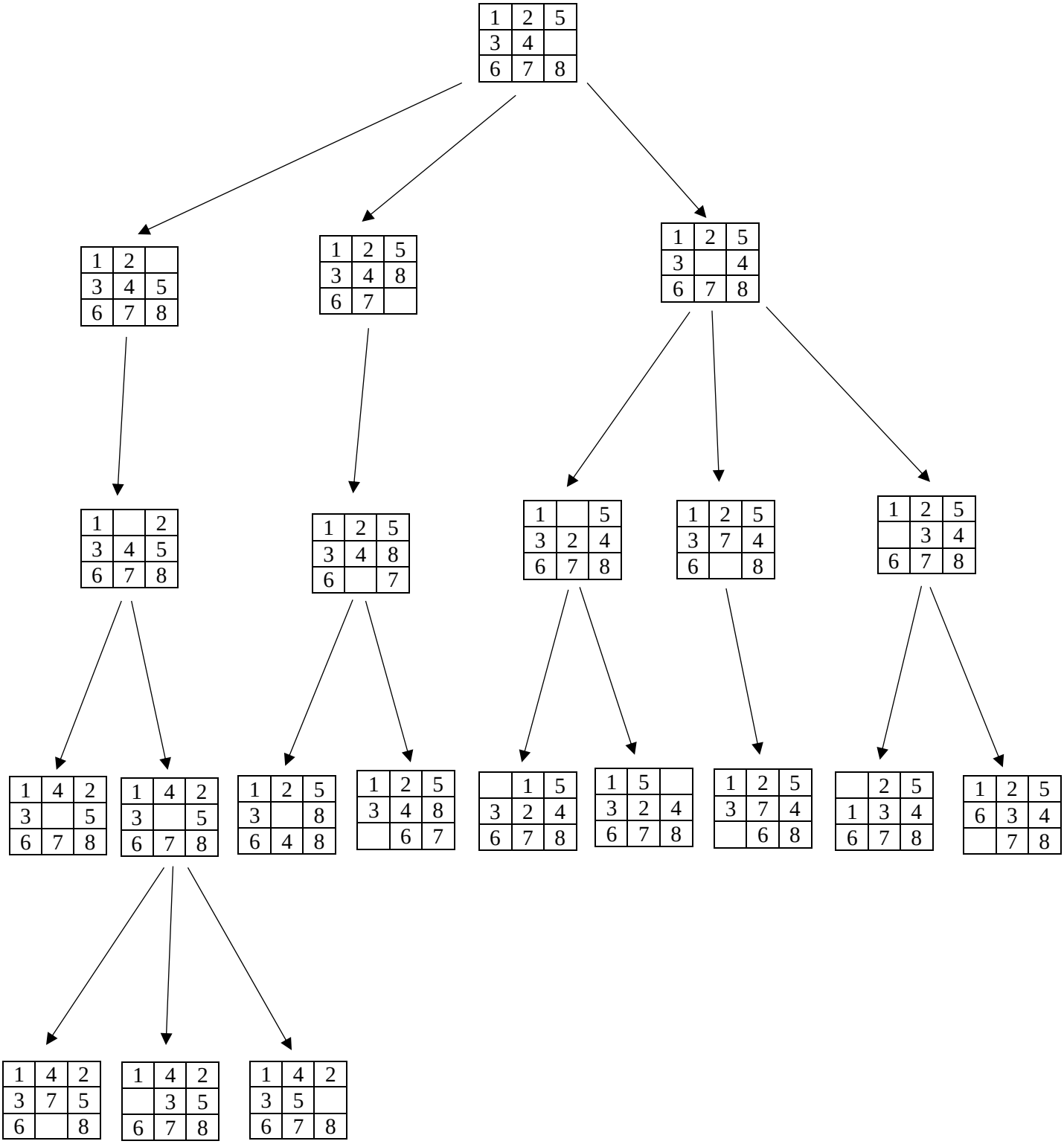


Figure 5.1 BFS State Space Tree

Code

```
#Import the necessary libraries
from time import time
```

```
from queue import Queue

#Creating a class Puzzle
class Puzzle:
    #Setting the goal state of 8-puzzle
    goal_state=[1,2,3,8,0,4,7,6,5]
    num_of_instances=0
    #default constructor to initialize the class members
    def __init__(self,state,parent,action):
        self.parent=parent
        self.state=state
        self.action=action   #action is used to generate current state from parent state

        #TODO: incrementing the number of instance by 1
        # num_of_instances = num_of_instances + 1
        Puzzle.num_of_instances+= 1
        print("Current state: ", Puzzle.num_of_instances)
        print(self.state)

    #function used to display a state of 8-puzzle
    def __str__(self):
        return str(self.state[0:3])+'\n'+str(self.state[3:6])+'\n'+str(self.state[6:9])

    #method to compare the current state with the goal state
    def goal_test(self):
        #TODO: include a condition to compare the current state with the goal state
        if self.state == Puzzle.goal_state:
            print("Goal state found and printed in reverse order: ")

print(str(Puzzle.goal_state[0:3])+'\n'+str(Puzzle.goal_state[3:6])+'\n'+str(Puzzle.goal_state[6:9]))
            return True
        else:
            return False

    #static method to find the legal action based on the current board position
    @staticmethod
    def find_legal_actions(i,j):
        legal_action = ['U', 'D', 'L', 'R']
        if i == 0:
            # if row is 0 in board then up is disable
            legal_action.remove('U')
        elif i == 2:
            #TODO: down is disable
            legal_action.remove('D')
```

```
    if j == 0:
        #TODO: Left is disable
        legal_action.remove('L')
    elif j == 2:
        #TODO: Right is disable
        legal_action.remove('R')
    return legal_action

#method to generate the child of the current state of the board
def generate_child(self):
    #TODO: create an empty list
    children=[]
    x = self.state.index(0)
    i = int(x / 3)
    j = int(x % 3)
    #TODO: call the method to find the legal actions based on i and j values
    legal_actions=Puzzle.find_legal_actions(i, j)

    #TODO:Iterate over all legal actions
    for action in legal_actions:
        new_state = self.state.copy()
        #if the legal action is UP
        if action == 'U':
            #Swapping between current index of 0 with its up element on the board
            new_state[x], new_state[x-3] = new_state[x-3], new_state[x]
        elif action == 'D':
            #TODO: Swapping between current index of 0 with its down element on the board
            new_state[x], new_state[x+3] = new_state[x+3], new_state[x]
        elif action == 'L':
            #TODO: Swapping between the current index of 0 with its left element on the board
            new_state[x], new_state[x-1] = new_state[x-1], new_state[x]
        elif action == 'R':
            #TODO: Swapping between the current index of 0 with its right element on the board
            new_state[x], new_state[x+1] = new_state[x+1], new_state[x]
        children.append(Puzzle(new_state,self,action))
    #TODO: return the children
    return children


    #TODO:Iterate over all legal actions
    for action in legal_actions:
        new_state = self.state.copy()
        #if the legal action is UP
        if action == 'U':
            #Swapping between current index of 0 with its up element on the board
```

```
            new_state[x], new_state[x-3] = new_state[x-3], new_state[x]
        elif action == 'D':
            #TODO: Swapping between current index of 0 with its down element on the board
            new_state[x], new_state[x+3] = new_state[x+3], new_state[x]
        elif action == 'L':
            #TODO: Swapping between the current index of 0 with its left element on the board
            new_state[x], new_state[x-1] = new_state[x-1], new_state[x]
        elif action == 'R':
            #TODO: Swapping between the current index of 0 with its right element on the board
            new_state[x], new_state[x+1] = new_state[x+1], new_state[x]
        children.append(Puzzle(new_state,self,action))
    #TODO: return the children
    return children


  #method to find the solution
  def find_solution(self):
    solution = []
    solution.append(self.action)
    path = self
    while path.parent != None:
        path = path.parent
        print("  |  ")
        print(path)
        solution.append(path.action)
    solution = solution[:-1]
    solution.reverse()
    return solution


#method for breadth first search
#TODO: pass the initial_state as parameter to the breadth_first_search method
def breadth_first_search(initial_state):
  start_node = Puzzle(initial_state, None, None)
  print("Initial state:")
  print(start_node)
  print("STATES OF THR BOARD")
  if start_node.goal_test():
    return start_node.find_solution()
  q = Queue()
  #TODO: put start_node into the Queue
  q.put(start_node)
  #TODO: create an empty list of explored nodes
  explored=[]
  #TODO: Iterate the queue until empty. Use the empty() method of Queue
  while not(q.empty):
```

```
        #TODO: get the current node of a queue. Use the get() method of Queue
        node=q.get
        #TODO: Append the state of node in the explored list as node.state
        explored.append(node.state)
        #TODO: call the generate_child method to generate the child nodes of current node
        children= node.generate_child(node)
        #TODO: Iterate over each child node in children
        for child in children:
            if child.state not in explored:
                if child.goal_test():
                    return child.find_solution()
                q.put(child)
    return

#Start executing the 8-puzzle with setting up the initial state
#Here we have considered 3 initial state intitalized using state variable
state=[ 1, 2, 5,
        3, 4, 0,
        6, 7, 8],

#Iterate over number of initial_state
for i in range(len(state)):
    #TODO: Initialize the num_of_instances to zero
    Puzzle.num_of_instances=0
    #Set t0 to current time
    t0=time()
    bfs=breadth_first_search(state[i])
    #Get the time t1 after executing the breadth_first_search method
    t1=time()-t0
    print('BFS:', bfs)
    print('space:',Puzzle.num_of_instances)
    print('time:',t1)
```

Output

```
Current state:  1
[1, 2, 5, 3, 4, 0, 6, 7, 8]
Initial state:
[1, 2, 5]
[3, 4, 0]
[6, 7, 8]

STATES OF THE BOARD
```

```
Current state:  2
[1, 2, 0, 3, 4, 5, 6, 7, 8]
Current state:  3
[1, 2, 5, 3, 4, 8, 6, 7, 8]
Current state:  4
[1, 2, 5, 3, 0, 4, 6, 7, 8]
Current state:  5
[1, 2, 5, 3, 4, 0, 6, 7, 8]
Current state:  6
[1, 0, 2, 3, 4, 5, 6, 7, 8]
Current state:  7
[1, 2, 5, 3, 4, 0, 6, 7, 8]
Current state:  8
[1, 2, 5, 3, 4, 8, 6, 0, 8]
Current state:  9
[1, 0, 5, 3, 2, 4, 6, 7, 8]
Current state:  10
[1, 2, 5, 3, 7, 4, 6, 7, 8]
Current state:  11
[1, 2, 5, 0, 3, 4, 6, 7, 8]
Current state:  12
[1, 2, 5, 3, 4, 0, 6, 7, 8]
Current state:  13
[1, 4, 2, 3, 0, 5, 6, 7, 8]
Current state:  14
[0, 1, 2, 3, 4, 5, 6, 7, 8]
Current state:  15
[1, 2, 0, 3, 4, 5, 6, 7, 8]
Current state:  16
[1, 2, 5, 3, 0, 8, 6, 4, 7]
Current state:  17
[1, 2, 5, 3, 4, 8, 0, 6, 7]
Current state:  18
[1, 2, 5, 3, 4, 8, 6, 7, 0]
Current state:  19
[1, 2, 5, 3, 0, 4, 6, 7, 8]
Current state:  20
[0, 1, 5, 3, 2, 4, 6, 7, 8]
Current state:  21
[1, 5, 0, 3, 2, 4, 6, 7, 8]
Current state:  22
[1, 2, 5, 3, 0, 4, 6, 7, 8]
Current state:  23
[1, 2, 5, 3, 7, 4, 0, 6, 8]
```

```
Current state:  24
[1, 2, 5, 3, 7, 4, 6, 8, 0]
Current state:  25
[0, 2, 5, 1, 3, 4, 6, 7, 8]
Current state:  26
[1, 2, 5, 6, 3, 4, 0, 7, 8]
Current state:  27
[1, 2, 5, 3, 0, 4, 6, 7, 8]
Current state:  28
[1, 0, 2, 3, 4, 5, 6, 7, 8]
Current state:  29
[1, 4, 2, 3, 7, 5, 6, 0, 8]
Current state:  30
[1, 4, 2, 0, 3, 5, 6, 7, 8]
Current state:  31
[1, 4, 2, 3, 5, 0, 6, 7, 8]

BFS:[ 'U', 'L', 'D', 'R']
Space: 31
Time: 0.12287497520446777
```

Lab – 6 (A)

Aim: To implement tic-tac-toe problem using Minimax Algorithm

Theory:

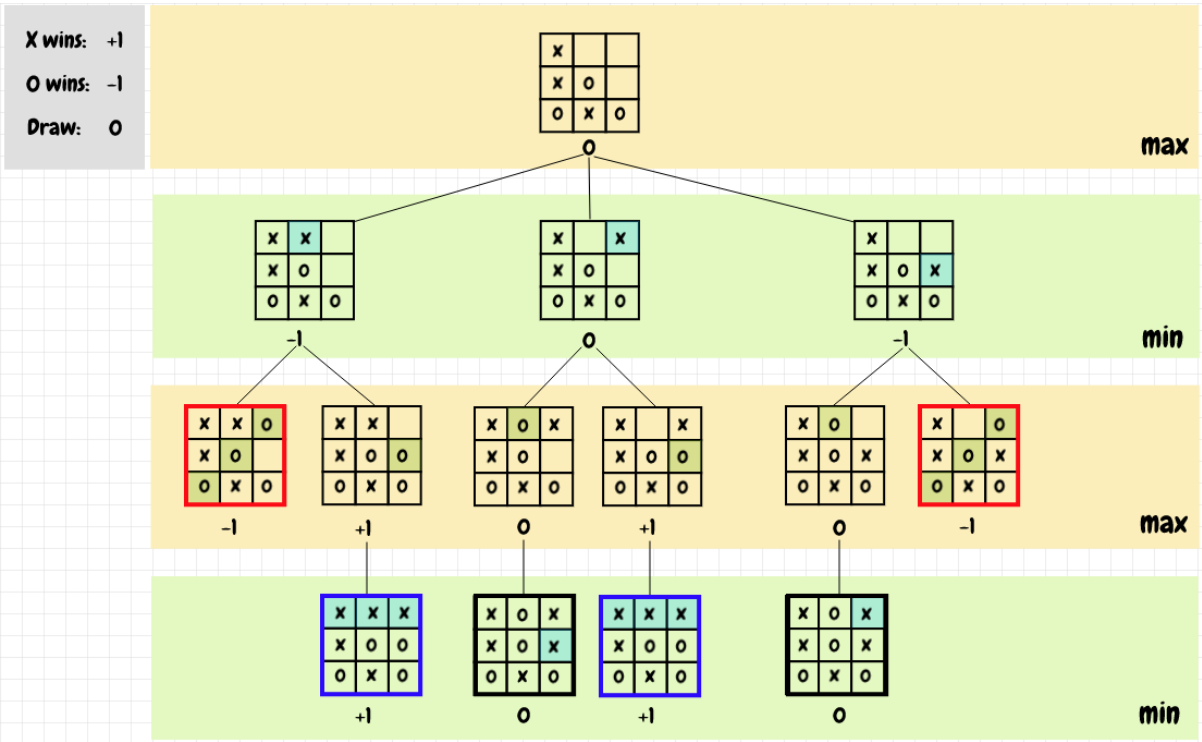
Tic-Tac-Toe Game using Minimax Algorithm

What is Minimax?

Minimax is a artificial intelligence applied in two player games, such as tic-tac-toe, checkers, chess and go. These games are known as zero-sum games, because in a mathematical representation: one player wins (+1) and other player loses (-1) or both of anyone not to win (0).

Minimax is a type of adversarial search algorithm for generating and exploring game trees. It is mostly used to solve zero-sum games where one side's gain is equivalent to other side's loss, so adding all gains and subtracting all losses end up being zero.

Adversarial search differs from conventional searching algorithms by adding opponents into the mix. Minimax algorithm keeps playing the turns of both player and the opponent optimally to figure out the best possible move.



How does it works?

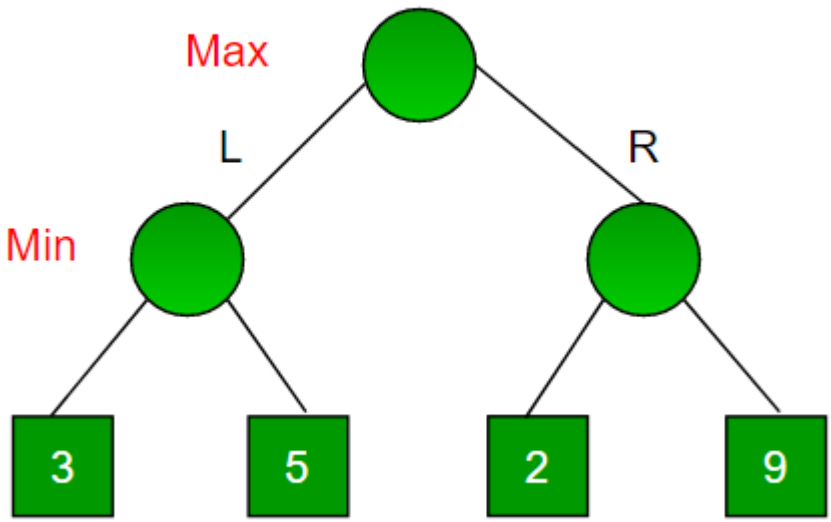
The algorithm search, recursively, the best move that leads the Max player to win or not lose (draw). It consider the current state of the game and the available moves at that state, then for each valid move it plays (alternating min and max) until it finds a terminal state (win, draw or lose).

Minimax is a kind of backtracking algorithm that is used in decision making and game theory to find the optimal move for a player, assuming that your opponent also plays optimally. It is widely used in two player turn-based games such as Tic-Tac-Toe, Backgammon, Mancala, Chess, etc.

In Minimax the two players are called maximizer and minimizer. The maximizer tries to get the highest score possible while the minimizer tries to do the opposite and get the lowest score possible.

Every board state has a value associated with it. In a given state if the maximizer has upper hand then, the score of the board will tend to be some positive value. If the minimizer has the upper hand in that board state then it will tend to be some negative value. The values of the board are calculated by some heuristics which are unique for every type of game.

Example: Consider a game which has 4 final states and paths to reach final state are from root to 4 leaves of a perfect binary tree as shown below. Assume you are the maximizing player and you get the first chance to move, i.e., you are at the root and your opponent at next level. Which move you would make as a maximizing player considering that your opponent also plays optimally?

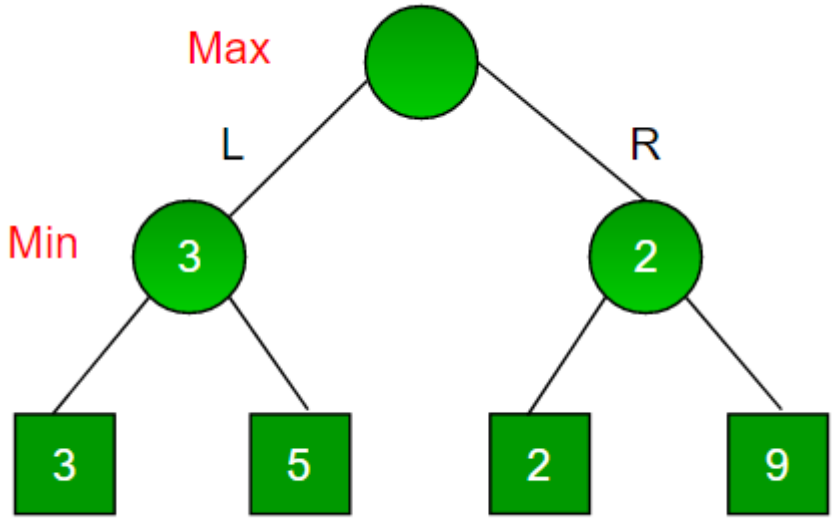


Since this is a backtracking based algorithm, it tries all possible moves, then backtracks and makes a decision.

Maximizer goes LEFT: It is now the minimizers turn. The minimizer now has a choice between 3 and 5. Being the minimizer it will definitely choose the least among both, that is 3
Maximizer goes RIGHT: It is now the minimizers turn. The minimizer now has a choice

between 2 and 9. He will choose 2 as it is the least among the two values. Being the maximizer you would choose the larger value that is 3. Hence the optimal move for the maximizer is to go LEFT and the optimal value is 3.

Now the game tree looks like below :



The above tree shows two possible scores when maximizer makes left and right moves.

Note: Even though there is a value of 9 on the right subtree, the minimizer will never pick that. We must always assume that our opponent plays optimally.

Code:

```
import numpy as np
from math import inf as infinity

game_state = [[' ',' ',' '],
              [' ',' ',' '],
              [' ',' ',' ']]
players = ['X','O']

def play_move(state, player, block_num):
    if state[int((block_num-1)/3)][(block_num-1)%3] is ' ':
        state[int((block_num-1)/3)][(block_num-1)%3] = player
    else:
        block_num = int(input("Block is not empty, ya blockhead! Choose again: "))
        play_move(state, player, block_num)

def copy_game_state(state):
    new_state = [[' ',' ',' '],[' ',' ',' '],[' ',' ',' ']]
    for i in range(3):
        for j in range(3):
```

```
            new_state[i][j] = state[i][j]
    return new_state

def check_current_state(game_state):
    # Check if draw
    draw_flag = 0
    for i in range(3):
        for j in range(3):
            if game_state[i][j] is ' ':
                draw_flag = 1
    if draw_flag is 0:
        return None, "Draw"

    # Check horizontals
    if (game_state[0][0] == game_state[0][1] and game_state[0][1] == game_state[0][2] and
game_state[0][0] is not ' '):
        return game_state[0][0], "Done"
    if (game_state[1][0] == game_state[1][1] and game_state[1][1] == game_state[1][2] and
game_state[1][0] is not ' '):
        return game_state[1][0], "Done"
    if (game_state[2][0] == game_state[2][1] and game_state[2][1] == game_state[2][2] and
game_state[2][0] is not ' '):
        return game_state[2][0], "Done"

    # Check verticals
    if (game_state[0][0] == game_state[1][0] and game_state[1][0] == game_state[2][0] and
game_state[0][0] is not ' '):
        return game_state[0][0], "Done"
    if (game_state[0][1] == game_state[1][1] and game_state[1][1] == game_state[2][1] and
game_state[0][1] is not ' '):
        return game_state[0][1], "Done"
    if (game_state[0][2] == game_state[1][2] and game_state[1][2] == game_state[2][2] and
game_state[0][2] is not ' '):
        return game_state[0][2], "Done"

    # Check diagonals
    if (game_state[0][0] == game_state[1][1] and game_state[1][1] == game_state[2][2] and
game_state[0][0] is not ' '):
        return game_state[1][1], "Done"
    if (game_state[2][0] == game_state[1][1] and game_state[1][1] == game_state[0][2] and
game_state[2][0] is not ' '):
        return game_state[1][1], "Done"

    return None, "Not Done"

def print_board(game_state):
    print('----------------')
```

```
    print('| ' + str(game_state[0][0]) + ' || ' + str(game_state[0][1]) + ' || ' + str(game_state[0][2])
+ ' |')
    print('----------------')
    print('| ' + str(game_state[1][0]) + ' || ' + str(game_state[1][1]) + ' || ' + str(game_state[1][2])
+ ' |')
    print('----------------')
    print('| ' + str(game_state[2][0]) + ' || ' + str(game_state[2][1]) + ' || ' + str(game_state[2][2])
+ ' |')
    print('----------------')
def getBestMove(state, player):
    '''
    Minimax Algorithm
    '''
    winner_loser , done = check_current_state(state)
    if done == "Done" and winner_loser == 'O': # If AI won
        return 1
    elif done == "Done" and winner_loser == 'X': # If Human won
        return -1
    elif done == "Draw":    # Draw condition
        return 0

    moves = []
    empty_cells = []
    for i in range(3):
        for j in range(3):
            if state[i][j] is ' ':
                empty_cells.append(i*3 + (j+1))

    for empty_cell in empty_cells:
        move = {}
        move['index'] = empty_cell
        new_state = copy_game_state(state)
        play_move(new_state, player, empty_cell)

        if player == 'O':    # If AI
            result = getBestMove(new_state, 'X')    # make more depth tree for human
            move['score'] = result
        else:
            result = getBestMove(new_state, 'O')    # make more depth tree for AI
            move['score'] = result

        moves.append(move)

    # Find best move
    best_move = None
    if player == 'O':   # If AI player
        best = -infinity
```

```
        for move in moves:
            if move['score'] > best:
                best = move['score']
                best_move = move['index']
    else:
        best = infinity
        for move in moves:
            if move['score'] < best:
                best = move['score']
                best_move = move['index']

    return best_move

# PLaying
play_again = 'Y'
while play_again == 'Y' or play_again == 'y':
    game_state = [[' ',' ',' '],
                  [' ',' ',' '],
                  [' ',' ',' ']]
    current_state = "Not Done"
    print("\nNew Game!")
    print_board(game_state)
    player_choice = input("Choose which player goes first - X (You - the petty human) or
O(The mighty AI): ")
    winner = None

    if player_choice == 'X' or player_choice == 'x':
        current_player_idx = 0
    else:
        current_player_idx = 1

    while current_state == "Not Done":
        if current_player_idx == 0: # Human's turn
            block_choice = int(input("Oye Human, your turn! Choose where to place (1 to 9): "))
            play_move(game_state ,players[current_player_idx], block_choice)
        else:   # AI's turn
            block_choice = getBestMove(game_state, players[current_player_idx])
            play_move(game_state ,players[current_player_idx], block_choice)
            print("AI plays move: " + str(block_choice))
        print_board(game_state)
        winner, current_state = check_current_state(game_state)
        if winner is not None:
            print(str(winner) + " won!")
        else:
            current_player_idx = (current_player_idx + 1)%2

        if current_state is "Draw":
```

```
        print("Draw!")

    play_again = input('Wanna try again?(Y/N) : ')
    if play_again == 'N':
        print('GG!')
```

Output:

```
New Game!
----------------
|   ||   ||   |
----------------
|   ||   ||   |
----------------
|   ||   ||   |
----------------
Choose which player goes first - X (You - the petty human) or O(The
mighty AI): x
Oye Human, your turn! Choose where to place (1 to 9): 5
----------------
|   ||   ||   |
----------------
|   || X ||   |
----------------
|   ||   ||   |
----------------
AI plays move: 2
----------------
|   || O ||   |
----------------
|   || X ||   |
----------------
|   ||   ||   |
----------------
Oye Human, your turn! Choose where to place (1 to 9): 3
----------------
|   || O || X |
----------------
|   || X ||   |
----------------
|   ||   ||   |
----------------
AI plays move: 7
----------------
|   || O || X |
----------------
|   || X ||   |
----------------
| O ||   ||   |
----------------
Oye Human, your turn! Choose where to place (1 to 9): 1
----------------
| X || O || X |
----------------
|   || X ||   |
----------------
```

```
| O ||   ||   |
----------------
AI plays move: 4
----------------
| X || O || X |
----------------
| O || X ||   |
----------------
| O ||   ||   |
----------------
Oye Human, your turn! Choose where to place (1 to 9): 9
----------------
| X || O || X |
----------------
| O || X ||   |
----------------
| O ||   || X |
----------------
X won!
Wanna try again? (Y/N): N
GG!
```

Criteria	Total Marks	Marks Obtained	Comments
Concept(A)	2		
Implementation(B)	2		
Performance	2		
Total	6 (to be scaled down to 1)		

Lab – 6 (B)

Aim: To implement tic-tac-toe problem using Alpha-Beta Pruning

Theory:

Alpha Beta Pruning

- Alpha-beta pruning is a modified version of the minimax algorithm. It is an optimization technique for the minimax algorithm.
- As we have seen in the minimax search algorithm that the number of game states it has to examine are exponential in depth of the tree. Since we cannot eliminate the exponent, but we can cut it to half. Hence there is a technique by which without checking each node of the game tree we can compute the correct minimax decision, and this technique is called **pruning**. This involves two threshold parameter Alpha and beta for future expansion, so it is called **alpha-beta pruning**. It is also called as **Alpha-Beta Algorithm**.
- Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prune the tree leaves but also entire sub-tree.
- The two-parameter can be defined as:
    1. **Alpha:** The best (highest value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is $-\infty$.
    2. **Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is $+\infty$.
- The Alpha-beta pruning to a standard minimax algorithm returns the same move as the standard algorithm does, but it removes all the nodes which are not really affecting the final decision but making algorithm slow. Hence by pruning these nodes, it makes the algorithm fast.

Condition for Alpha-Beta Pruning:

The main condition which required for alpha-beta pruning is:

$$\alpha >= \beta$$

Key points about alpha-beta pruning:

- The Max player will only update the value of alpha.
- The Min player will only update the value of beta.

- While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta.
- We will only pass the alpha, beta values to the child nodes.

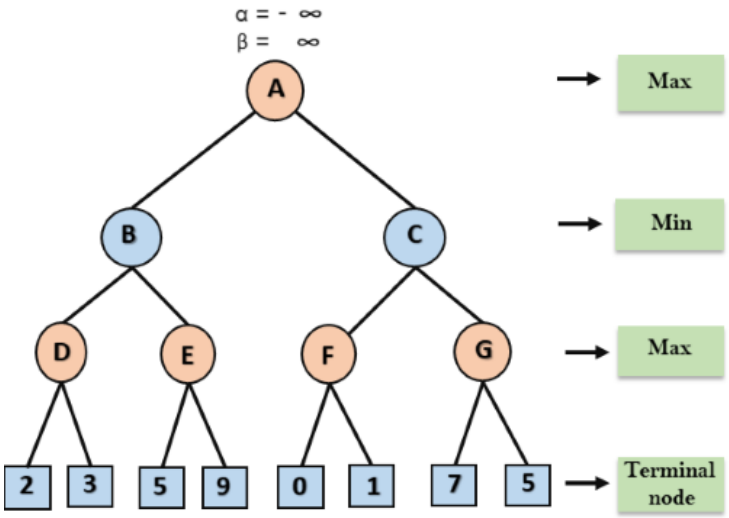
Pseudo-code for Alpha-beta Pruning:

```
function minimax (node, depth, alpha, beta, maximizing Player) is
if depth ==0 or node is a terminal node then
return static evaluation of node

if Maximizing Player then      // for Maximizer Player
   max Eva= -infinity
   for each child of node do
   eva= minimax (child, depth-1, alpha, beta, False)
  Max Eva= max(max Eva, Eva)
 alpha= max(alpha, maxEva)
  if beta<=alpha
 break
 return maxEva

else                           // for Minimizer player
   minEva= +infinity
   for each child of node do
   eva= minimax(child, depth-1, alpha, beta, true)
   minEva= min(minEva, eva)
   beta= min(beta, eva)
    if beta<=alpha
  break
 return minEva
```

Working of Alpha-Beta Pruning:

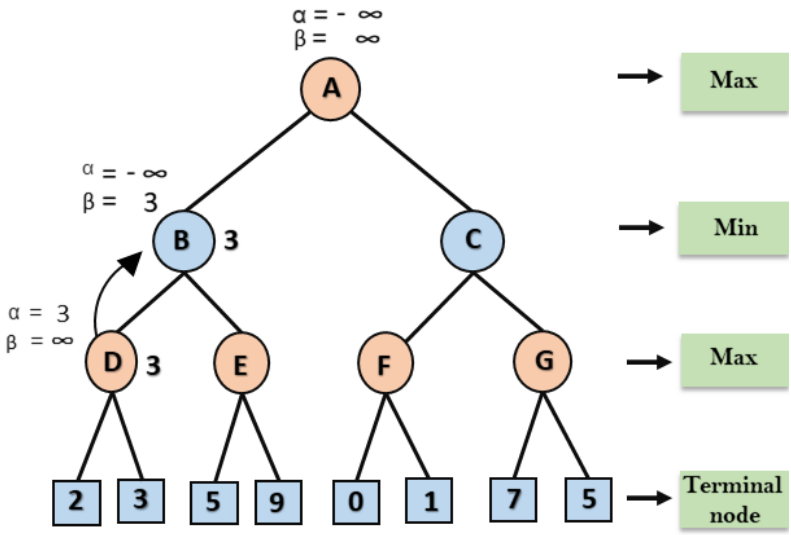
Let's take an example of two-player search tree to understand the working of Alpha-beta pruning

Step 1: At the first step the, Max player will start first move from node A where $\alpha = -\infty$ and $\beta = +\infty$, these value of alpha and beta passed down to node B where again $\alpha = -\infty$ and $\beta = +\infty$, and Node B passes the same value to its child D.



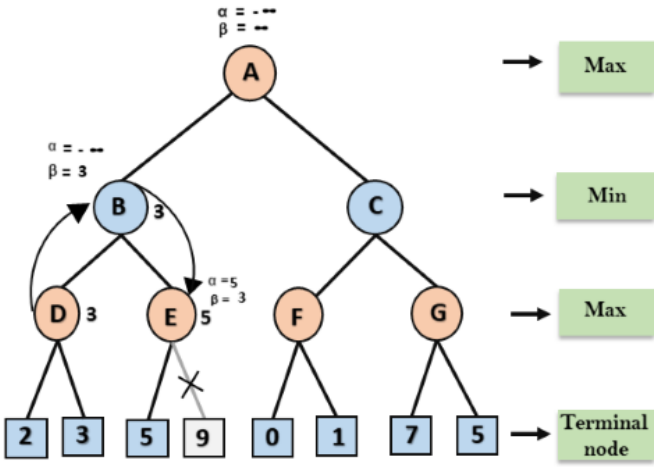
Step 2: At Node D, the value of α will be calculated as its turn for Max. The value of α is compared with firstly 2 and then 3, and the max (2, 3) = 3 will be the value of α at node D and node value will also 3.

Step 3: Now algorithm backtrack to node B, where the value of β will change as this is a turn of Min, Now β= +∞, will compare with the available subsequent nodes value, i.e. min (∞, 3) = 3, hence at node B now α= -∞, and β= 3.



In the next step, algorithm traverse the next successor of Node B which is node E, and the values of α= -∞, and β= 3 will also be passed.

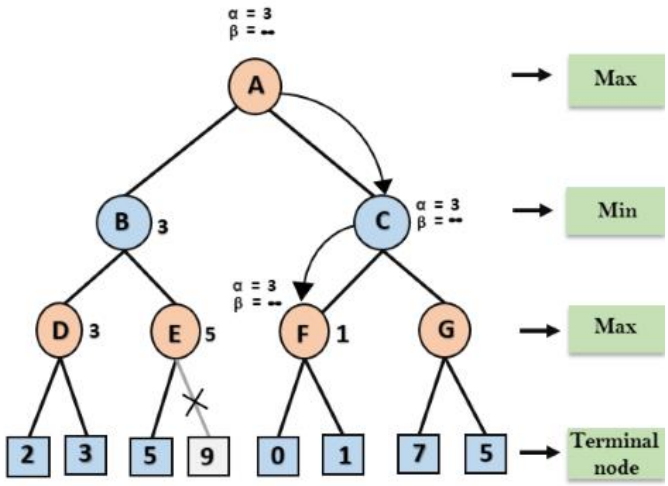
Step 4: At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so max (-∞, 5) = 5, hence at node E α= 5 and β= 3, where α>=β, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5.



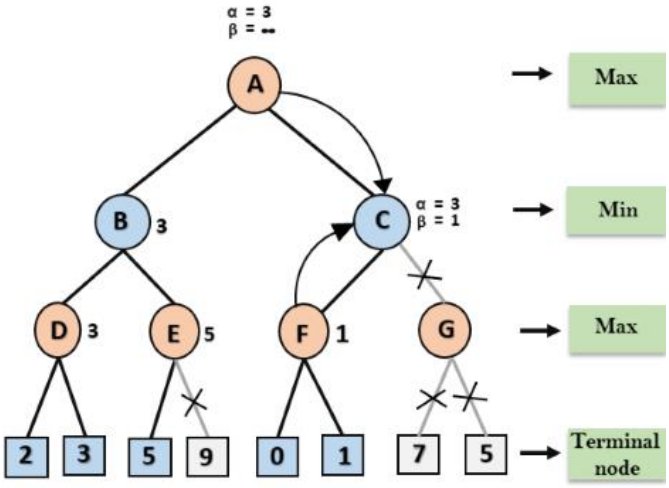
Step 5: At next step, algorithm again backtrack the tree, from node B to node A. At node A, the value of alpha will be changed the maximum available value is 3 as max (-∞, 3)= 3, and β= +∞, these two values now passes to right successor of A which is Node C.

At node C, α=3 and β= +∞, and the same values will be passed on to node F.

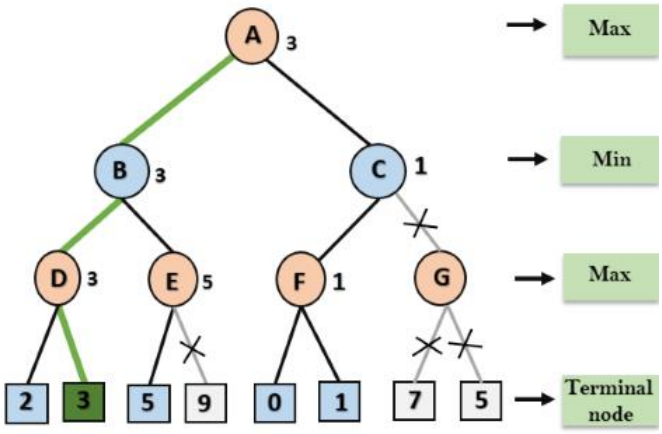
Step 6: At node F, again the value of α will be compared with left child which is 0, and max(3,0)= 3, and then compared with right child which is 1, and max(3,1)= 3 still α remains 3, but the node value of F will become 1.



Step 7: Node F returns the node value 1 to node C, at C α= 3 and β= +∞, here the value of beta will be changed, it will compare with 1 so min (∞, 1) = 1. Now at C, α=3 and β= 1, and again it satisfies the condition α>=β, so the next child of C which is G will be pruned, and the algorithm will not compute the entire sub-tree G.



Step 8: C now returns the value of 1 to A here the best value for A is max (3, 1) = 3. Following is the final game tree which is the showing the nodes which are computed and nodes which has never computed. Hence the optimal value for the maximizer is 3 for this example.



Code:

```
#Import the necessary libraries
import numpy as np
from math import inf as infinity

#Set the Empty Board
game_state = [[' ',' ',' '],
              [' ',' ',' '],
              [' ',' ',' ']]
#Create the Two Players as 'X'/'O'
players = ['X','O']
pruned=0

#Method for checking the correct move on Tic-Tac-Toe
def play_move(state, player, block_num):
    if state[int((block_num-1)/3)][(block_num-1)%3] == ' ':
        #TODO: Assign the player move on the current position of Tic-Tac-
Toe if condition is True
        state[int((block_num-1)/3)][(block_num-1)%3] = player
    else:
        block_num = int(input("Block is not empty, ya blockhead! Choose again: "))
        play_move(state, player, block_num)
        #TODO: Recursively call the play_move

#Method to copy the current game state to new_state of Tic-Tac-Toe
def copy_game_state(state):
    new_state = [[' ',' ',' '],[' ',' ',' '],[' ',' ',' ']]
    for i in range(3):
        for j in range(3):
            #TODO: Copy the Tic-Tac-Toe state to new_state
            new_state[i][j] = state[i][j]
    #TODO: Return the new_state
    return new_state

#Method to check the current state of the Tic-Tac-Toe
def check_current_state(game_state):
    #TODO: Set the draw_flag to 0
    draw_flag = 0
    for i in range(3):
        for j in range(3):
            if game_state[i][j] == ' ':
                draw_flag = 1

    if draw_flag == 0:
        return None, "Draw"

    # Check horizontals in first row
```

```
    if (game_state[0][0] == game_state[0][1] and game_state[0][1] == game_state[0][2] and ga
me_state[0][0] != ' '):
        return game_state[0][0], "Done"
    #TODO: Check horizontals in second row
    if (game_state[1][0] == game_state[1][1] and game_state[1][1] == game_state[1][2] and ga
me_state[1][0] != ' '):
        return game_state[1][0], "Done"
    #TODO: Check horizontals in third row
    if (game_state[2][0] == game_state[2][1] and game_state[2][1] == game_state[2][2] and ga
me_state[2][0] != ' '):
        return game_state[2][0], "Done"

    # Check verticals in first column
    if (game_state[0][0] == game_state[1][0] and game_state[1][0] == game_state[2][0] and ga
me_state[0][0] != ' '):
        return game_state[0][0], "Done"
    # Check verticals in second column
    if (game_state[0][1] == game_state[1][1] and game_state[1][1] == game_state[2][1] and ga
me_state[0][1] != ' '):
        return game_state[0][1], "Done"
    # Check verticals in third column
    if (game_state[0][2] == game_state[1][2] and game_state[1][2] == game_state[2][2] and ga
me_state[0][2] != ' '):
        return game_state[0][2], "Done"

    # Check left diagonal
    if (game_state[0][0] == game_state[1][1] and game_state[1][1] == game_state[2][2] and ga
me_state[0][0] != ' '):
        return game_state[1][1], "Done"
    # Check right diagonal
    if (game_state[2][0] == game_state[1][1] and game_state[1][1] == game_state[0][2] and ga
me_state[2][0] != ' '):
        return game_state[1][1], "Done"

    return None, "Not Done"

#Method to print the Tic-Tac-Toe Board
def print_board(game_state):
    print('----------------')
    print('| ' + str(game_state[0][0]) + ' || ' + str(game_state[0][1]) + ' || ' + str(game_state[0][2])
 + ' |')
    print('----------------')
    print('| ' + str(game_state[1][0]) + ' || ' + str(game_state[1][1]) + ' || ' + str(game_state[1][2])
 + ' |')
    print('----------------')
    print('| ' + str(game_state[2][0]) + ' || ' + str(game_state[2][1]) + ' || ' + str(game_state[2][2])
 + ' |')
    print('----------------')
```

```
#Method for implement the alpha beta pruning function
def alphabeta(state,depth,alpha,beta,player):
    global pruned
    #TODO: call the check_current_state method using state parameter
    winner_loser , done = check_current_state(state)
    #TODO:Check condition for winner, if winner_loser is 'O' then Computer won
    #else if winner_loser is 'X' then You won else game is draw
    if done == "Done" and winner_loser == 'O': # If AI won
        return (1,0)
    elif done == "Done" and winner_loser == 'X': # If Human won
        return (-1,0)
    elif done == "Draw":    # Draw condition
        return (0,0)
    #TODO: set moves to empty list
    moves = []
    #TODO: set empty_cells to empty list
    empty_cells = []
    #Append the block_num to the empty_cells list
    for i in range(3):
        for j in range(3):
            if state[i][j] == ' ':
                empty_cells.append(i*3 + (j+1))

    #TODO:Iterate over all the empty_cells
    for empty_cell in empty_cells:
        #TODO: create the empty dictionary
        move = {}
        #TODO: Assign the empty_cell to move['index']
        move['index'] = empty_cell

        #Call the copy_game_state method
        new_state = copy_game_state(state)

        #TODO: Call the play_move method with new_state,player,empty_cell
        play_move(new_state, player, empty_cell)

        #if player is computer
        if player == 'O':
            #TODO: Call getBestMove method with new_state and human player ('X') to make m
ore depth tree for human
            result = alphabeta(new_state, depth-1, alpha, beta,False)[1]
            move['score'] = result
        else:
            #TODO: Call getBestMove method with new_state and computer player('O') to make
more depth tree for computer
            result = alphabeta(new_state, depth-1, alpha, beta,True)[1]
            move['score'] = result
```

```
        moves.append(move)
    # Find best move
    best_move = None
    #Check if player is computer('O')
    if player == "O":
        #TODO: Set best as -infinity for computer
        best = - infinity
        for move in moves:
            #TODO: Check if move['score'] is greater than best
            if move['score'] > best:
                best = move['score']
                best_move = move['index']
            alpha=max(alpha, best)
            if alpha >= beta:
                pruned+=(3-depth)**2
                # Increment pruned counter
                break
    else:
        #TODO: Set best as infinity for human
        best = infinity
        for move in moves:
        #TODO: Check if move['score'] is less than best
            if move['score'] < best:
                best = move['score']
                best_move = move['index']
            beta=min(alpha, best)
            if alpha >= beta:
                pruned+=(3-depth)**2
                break
    return (best, best_move,pruned)
# Now PLaying the Tic-Tac-Toe Game
play_again = 'Y'
while play_again == 'Y' or play_again == 'y':
    depth = 9
    #Set the empty board for Tic-Tac-Toe
    game_state = [[' ',' ',' '],
                  [' ',' ',' '],
                  [' ',' ',' ']]
    pruned=0
    #Set current_state as "Not Done"
    current_state = "Not Done"
    print("\nNew Game!")
    #print the game_state
    print_board(game_state)
    #Select the player_choice to start the game
    player_choice = input("Choose which player goes first - X (You) or O(Computer): ")
    #Set winner as None
    winner = None
```

```
    #if player_choice is ('X' or 'x') for humans else for computer
    if player_choice == 'X' or player_choice == 'x':
        #TODO: Set current_player_idx is 0
        current_player_idx = 0
    else:
        #TODO: Set current_player_idx is 1
        current_player_idx = 1
    while current_state == "Not Done":
        #For Human Turn
        if current_player_idx == 0:
            block_choice = int(input("Your turn please! Choose where to place (1 to 9): "))
            #TODO: Call the play_move with parameters as game_state ,players[current_player_i
dx], block_choice
            play_move(game_state ,players[current_player_idx], block_choice)
        else:
            best_move, best_score, pruned = alphabeta(game_state, depth, float('-
inf'), float('inf'),True)
            play_move(game_state ,players[current_player_idx], best_move)
            print(f"Best move: {best_move}, score: {best_score}, pruned: {pruned}")
            print("AI plays move: " + str(best_move))
            # Computer turn
#           _,block_choice = getBestMove(game_state,float('-
inf'),float('inf'),players[current_player_idx],pruned_states)
            #TODO: Call the play_move with parameters as game_state ,players[current_player_i
dx], block_choice
        print_board(game_state)
        #TODO: Call the check_current_state function for game_state
        winner, current_state = check_current_state(game_state)
        if winner is not None:
            print(str(winner) + " won!")
        else:
            current_player_idx = (current_player_idx + 1)%2

        if current_state == "Draw":
            print("Draw!")
    play_again = input('Wanna try again?(Y/N) : ')
    if play_again == 'N':
        print('Thank you for playing Tic-Tac-Toe Game!!!!!!')
```

Output:

```
New Game!
----------------
|   ||   ||   |
----------------
|   ||   ||   |
----------------
|   ||   ||   |
----------------
Choose which player goes first - X (You) or O(Computer): x
```

```
Your turn please! Choose where to place (1 to 9): 5
----------------
|   ||   ||   |
----------------
|   || X ||   |
----------------
|   ||   ||   |
----------------
Best move: 2, score: 1, pruned: 57916
AI plays move: 2
----------------
|   || O ||   |
----------------
|   || X ||   |
----------------
|   ||   ||   |
----------------
Your turn please! Choose where to place (1 to 9): 1
----------------
| X || O ||   |
----------------
|   || X ||   |
----------------
|   ||   ||   |
----------------
Best move: 4, score: 3, pruned: 61822
AI plays move: 4
----------------
| X || O ||   |
----------------
| O || X ||   |
----------------
|   ||   ||   |
----------------
Your turn please! Choose where to place (1 to 9): 9
----------------
| X || O ||   |
----------------
| O || X ||   |
----------------
|   ||   || X |
----------------
X won!
Wanna try again? (Y/N) : N

Thank you for playing Tic-Tac-Toe Game!!!!!!!
```

Criteria	Total Marks	Marks Obtained	Comments
Concept(A)	2		
Implementation(B)	2		
Performance	2		
Total	6 (to be scaled down to 1)		

LAB-7

Date: 09/03/2023

AIM: Consider the equation APPLE + LEMON = BANANA. Assume that each letter represents a digit from 0 to 9. Some conditions are imposed. The leftmost letter can't be zeroin any word. There must be a one-to-one mapping between letters and digits. In other words,if you choose the digit 5 for the letter E, then all the E's in the equation must be 5 and no other letter can be a 5. No digit can be repeated.

Theory:

In the crypt-arithmetic problem, some letters are used to assign digits to it. Like ten different letters are holding digit values from 0 to 9 to perform arithmetic operations correctly. There are two words are given and another word is given an answer of addition for those two words.

Code:

```
import itertools
def get_value(word,
    substitution):s = 0
    factor = 1
    for letter in reversed(word):
        s += factor *
        substitution[letter]factor *=
        10
    return s

def solve2(equation):
    # split equation in left and right
    left, right = equation.lower().replace(' ',
    '').split('=')# split words in left part
    left = left.split('+')
    # create list of used
    lettersletters =
    set(right)
    for word in left:
        for letter in
            word:
            letters.add(lett
            er)
    letters =
    list(letters)digits
    = range(10)
    for perm in itertools.permutations(digits,
```

```
        len(letters)):sol = dict(zip(letters, perm))
        if sum(get_value(word, sol) for word in left) == get_value(right, sol):
            print(' + '.join(str(get_value(word, sol)) for word in left) + " = {} (mapping:
{})".format(get_value(right, sol), sol))
solve2('APPLE+LEMON=BANANA')
```

Output:

67794 + 94832 = 162626 (mapping: {'p': 7, 'a': 6, 'b': 1, 'm': 8, 'n': 2, 'o': 3, 'e': 4, 'l': 9})

Criteria	Total Marks	Marks Obtained	Comments
Concept(A)	2		
Implementation(B)	2		
Performance	2		
Total	6 (to be scaled down to 1)		

Lab- 8

Aim- To implement graph colouring problem in python.

Theory

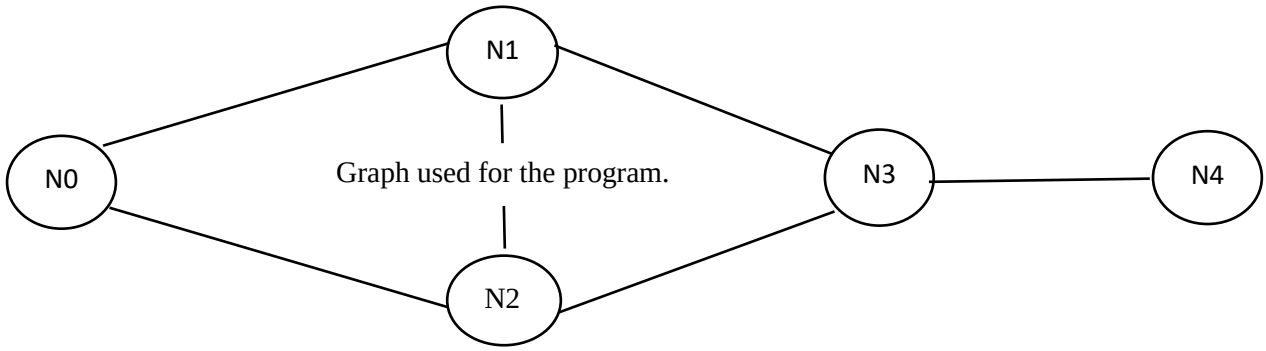
The graph colouring problem is a classical problem in computer science and mathematics that involves assigning colours to the vertices of a graph in such a way that no two adjacent vertices have the same colour. The greedy algorithm is a simple algorithm that can be used to solve this problem.

The greedy algorithm for the graph colouring problem works as follows:

1. Sort the vertices of the graph in some order.
2. Initialize an array of colours, where the colour of each vertex is initially set to 0 (i.e., no colour).
3. For each vertex v in the sorted order:
    a. Consider the colours of its neighbours.
    b. Choose the smallest colour that is not used by any of its neighbours.
    c. Assign that colour to v.
4. Return the array of colours.

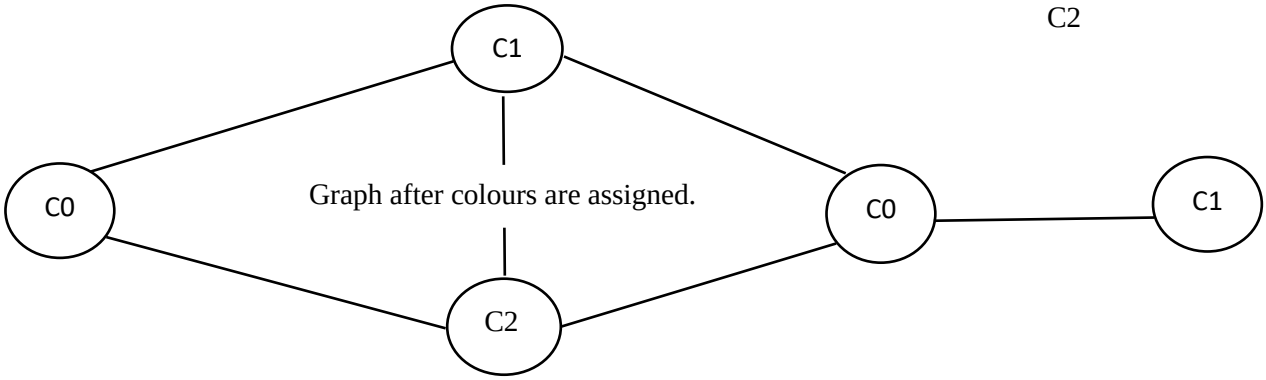
The backtracking algorithm for the graph colouring problem works as follows:

1. Choose an uncoloured vertex v.
2. For each possible colour c that can be assigned to v:
    a. Check if c is a valid colour for v (i.e., no adjacent vertex of v has colour c).
    b. If c is a valid colour for v, assign colour c to v and recursively apply the same steps to the next uncoloured vertex.
    c. If no valid colour can be found for v, backtrack (i.e., undo the colour assignment to v) and try a different colour for the previous vertex.
3. If all vertices are coloured, the algorithm has found a valid colour assignment.



Graph used for the program.

Colours used:
C0
C1
C2



Graph after colours are assigned.

Code:

```
def addEdge(adj, v, w):
  adj[v].append(w)
  adj[w].append(v)                    # The graph is undirected
  return adj

# Assigns colors (starting from 0) to all vertices and prints the assignment of colors
def greedyColoring(adj, V):
  result = [-1] * V
  result[0] = 0                       # Assign the first color to first vertex

# A temporary array to store the available colors.
# True value of available[cr] would mean that the color cr is assigned to one of its adjacent ve
rtices
  available = [False] * V

# Assign colors to remaining V-1 vertices
  for u in range(1, V):

    # Process all adjacent vertices and flag their colors as unavailable
    for i in adj[u]:
      if (result[i] != -1):
        available[result[i]] = True

    # Find the first available color
    cr = 0
    while cr < V:
      if (available[cr] == False):
        break
      cr += 1

    # Assign the found color
    result[u] = cr
    # Reset the values back to false for the next iteration
    for i in adj[u]:
      if (result[i] != -1):
        available[result[i]] = False

  # Print the result
  for u in range(V):
    print("Node", u, " ---> Color", result[u])
```

```
# Driver Code
if __name__ == '__main__':
  g1 = [[] for i in range(5)]
  g1 = addEdge(g1, 0, 1)
  g1 = addEdge(g1, 0, 2)
  g1 = addEdge(g1, 1, 2)
  g1 = addEdge(g1, 1, 3)
  g1 = addEdge(g1, 2, 3)
  g1 = addEdge(g1, 3, 4)
  print("Coloring of graph: ")
  greedyColoring(g1, 5)
```

Output:

```
Coloring of graph:
Node 0  ---> Color 0
Node 1  ---> Color 1
Node 2  ---> Color 2
Node 3  ---> Color 0
Node 4  ---> Color 1
```

Criteria	Total Marks	Marks Obtained	Comments
Concept(A)	2		
Implementation(B)	2		
Performance	2		
Total	6 (to be scaled down to 1)		

Lab-9

Aim- Write a program for tokenization of word and sentence using NLTK package in python.

Also perform:

1. Stop word removal
2. Stemming
3. Lemmatization
4. POS tagging (parsing)
5. Parsing of a sentence

Theory

The Natural Language Toolkit (NLTK) is a popular open-source Python library used for natural language processing (NLP) tasks such as tokenization, stemming, lemmatization, part-of-speech tagging, named entity recognition, parsing, and semantic analysis. NLTK provides a comprehensive set of tools and resources for processing and analysing human language data.

Tokenization is essentially splitting a phrase, sentence, paragraph, or an entire text document into smaller units, such as individual words or terms. Each of these smaller units are called tokens. The tokens could be words, numbers or punctuation marks. In tokenization, smaller units are created by locating word boundaries. These are the ending point of a word and the beginning of the next word.

Stop words are commonly used words that are filtered out from natural language processing tasks like text analysis and information retrieval. These words are considered insignificant and do not add meaning to the content of a document or sentence. Stop words typically include pronouns, prepositions, conjunctions, and other frequently occurring words in a language, such as "the", "and", "a", "an", "in", "on", "of", "to", etc.

Stemming is a text processing task in which words can be reduce to their root, which is the core part of a word. For example, the words “helping” and “helper” share the root “help.” Stemming allows you to zero in on the basic meaning of a word rather than all the details of how it’s being used. Types of stemming:

1. **Understemming** happens when two related words should be reduced to the same stem but aren’t. This is a false negative.
2. **Overstemming** happens when two unrelated words are reduced to the same stem even though they shouldn’t be. This is a false positive.

Lemmatization reduces words to their core meaning, but it will give a complete English word that makes sense on its own instead of just a fragment of a word like 'discoveri'.

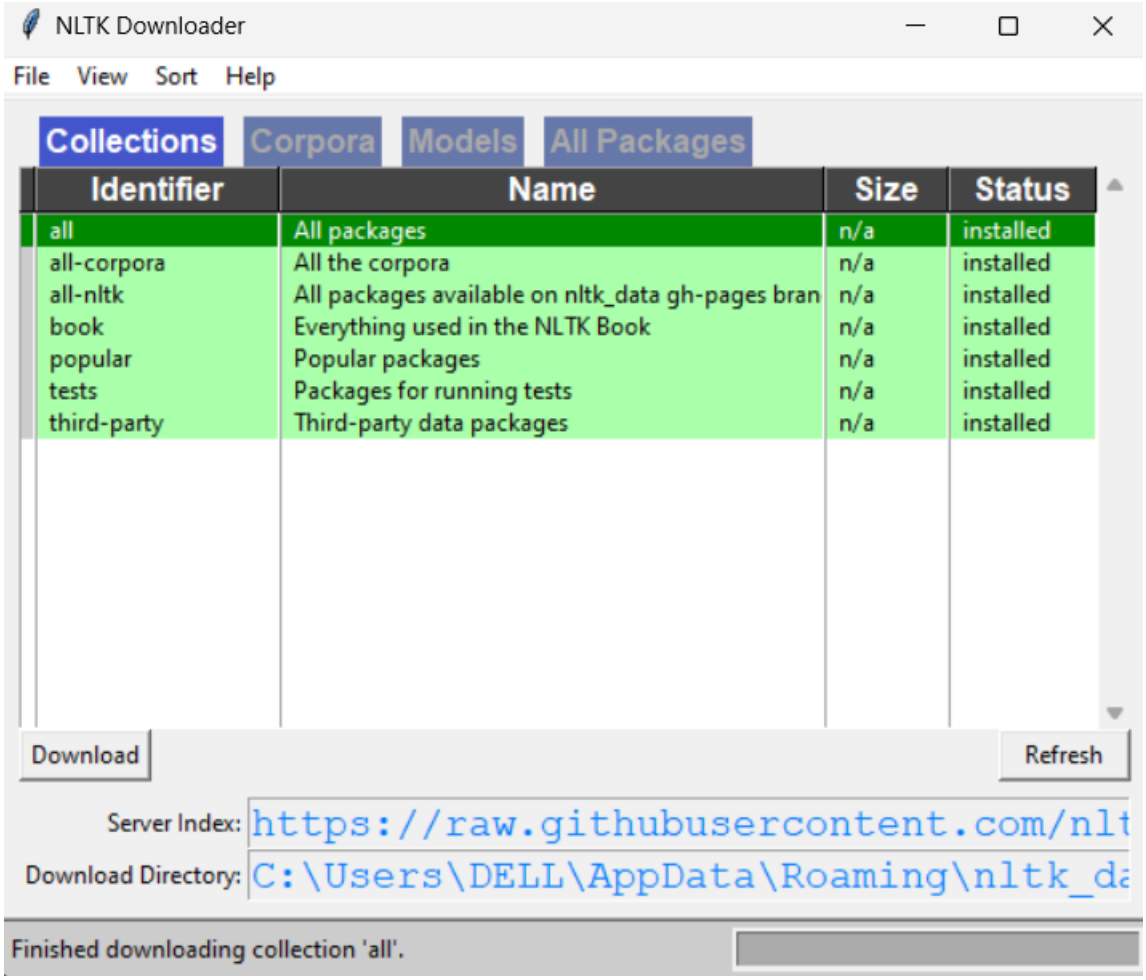
Lemma is a word that represents a whole group of words, and that group of words is called a **lexeme**. For example, if you were to look up the word “blending” in a dictionary, then you’d need to look at the entry for “blend,” but you would find “blending” listed in that entry. In this example, “blend” is the lemma, and “blending” is part of the lexeme.

Parts of Speech Tagging(POS) is a process to mark up the words in text format for a particular part of a speech based on its definition and context. It is responsible for text reading in a

language and assigning some specific token (Parts of Speech) to each word. It is also called grammatical tagging.

```
Microsoft Windows [Version 10.0.22623.870]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DELL>pip install nltk
```

```
>>> nltk.download()
...
showing info https://raw.githubusercontent.com/nltk/nltk_data/gh-pages/index.xml
```



Code

#Code for Tokenization of words and sentence

import nltk

from nltk.tokenize import sent_tokenize, word_tokenize

text = "Cryptarithmetic Problem is a type of constraint satisfaction problem where the game is about digits and its unique replacement " + \

"either with alphabets or other symbols. In cryptarithmetic problem, the digits (0-9) get substituted by some possible alphabets or symbols. " + \

"The task in cryptarithmetic problem is to substitute each digit with an alphabet to get the result arithmetically correct. "

```
nltk_tokens = nltk.sent_tokenize(text)

print(word_tokenize(text))

print("\n")

print(sent_tokenize(text))
```

Output

```
IDLE Shell 3.11.1
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec  6 2022, 19:58:39) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
================== RESTART: C:\Users\sumit\Desktop\Lab 9.py ==================
['Cryptarithmetic', 'Problem', 'is', 'a', 'type', 'of', 'constraint', 'satisfaction', 'problem',
 'where', 'the', 'game', 'is', 'about', 'digits', 'and', 'its', 'unique', 'replacement', 'either
', 'with', 'alphabets', 'or', 'other', 'symbols', '.', 'In', 'cryptarithmetic', 'problem', ',',
'the', 'digits', '(', '0-9', ')', 'get', 'substituted', 'by', 'some', 'possible', 'alphabets', '
or', 'symbols', '.', 'The', 'task', 'in', 'cryptarithmetic', 'problem', 'is', 'to', 'substitute'
, 'each', 'digit', 'with', 'an', 'alphabet', 'to', 'get', 'the', 'result', 'arithmetically', 'co
rrect', '.']


['Cryptarithmetic Problem is a type of constraint satisfaction problem where the game is about d
igits and its unique replacement either with alphabets or other symbols.', 'In cryptarithmetic p
roblem, the digits (0-9) get substituted by some possible alphabets or symbols.', 'The task in c
ryptarithmetic problem is to substitute each digit with an alphabet to get the result arithmetic
ally correct.']
```

```
#Code for stop word removal

import nltk

from nltk.corpus import stopwords

nltk.download('stopwords')

from nltk.tokenize import word_tokenize

text = "Nick likes to play football, however he is not too fond of tennis."

text_tokens = word_tokenize(text)

tokens_without_sw = [word for word in text_tokens if not word in stopwords.words()]

print("After removing stop words:")

print(tokens_without_sw)
```

Output

```
IDLE Shell 3.11.1
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec  6 2022, 19:58:39) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
================== RESTART: C:\Users\sumit\Desktop\Lab 9.py ==================
After removing stop words:
['Nick', 'likes', 'play', 'football', ',', 'fond', 'tennis', '.']
```

```
#Code for stemming
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
ps = PorterStemmer()
words = ["program", "programs", "programmer", "programming", "programmers"]
sentence = "Stemming is a text processing task where words can be reduced to roots"
print("\nWord Stemming")
for w in words:
        print(w, " : ", ps.stem(w))

print("\nSentence Stemming")
words = word_tokenize(sentence)

for w in words:
   print(w, " : ", ps.stem(w))
```

Output

```
IDLE Shell 3.11.1
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec  6 2022, 19:58:39)
Type "help", "copyright", "credits" or "license()" for more
>>>
==================== RESTART: C:\Users\sumit\Desktop\Lab 9.py
Word Stemming
program  :  program
programs  :  program
programmer  :  programm
programming  :  program
programmers  :  programm

Sentence Stemming
Stemming  :  stem
is  :  is
a  :  a
text  :  text
processing  :  process
task  :  task
where  :  where
words  :  word
can  :  can
be  :  be
reduced  :  reduc
to  :  to
roots  :  root
```

```
#Code for Lemmatization
import nltk
from nltk.stem import WordNetLemmatizer
wordnet_lemmatizer = WordNetLemmatizer()
text = "The children were running and laughing in the playground."
tokenization = nltk.word_tokenize(text)
print("\n")
for w in tokenization:
    print("Lemma for {} is {}".format(w, wordnet_lemmatizer.lemmatize(w)))
```

Output

```
IDLE Shell 3.11.1
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec  6 2022, 19:58:39)
Type "help", "copyright", "credits" or "license()" for more
>>>
=================== RESTART: C:\Users\sumit\Desktop\Lab 9.py
Lemma for The is The
Lemma for children is child
Lemma for were is were
Lemma for running is running
Lemma for and is and
Lemma for laughing is laughing
Lemma for in is in
Lemma for the is the
Lemma for playground is playground
Lemma for . is .
```

```
#Code for POS tagging
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
stop_words = set(stopwords.words('english'))
txt ="Parts of Speech Tagging is a process to mark up the words in text " \
        "format for a particular part of a speech based on its definition " \
        "It is responsible for text reading in a language " \
        "It is also called grammatical tagging. "
tokenized = sent_tokenize(txt)
for i in tokenized:
    wordsList = nltk.word_tokenize(i)
#Using a POS Tagger.
tagged = nltk.pos_tag(wordsList)
print("\nPOS tagged text:")
print(tagged)
```

Output

```
POS tagged text:
[('Parts', 'NNS'), ('of', 'IN'), ('Speech', 'NNP'), ('Tagging', 'NNP'), ('is', 'VBZ'), ('
a', 'DT'), ('process', 'NN'), ('to', 'TO'), ('mark', 'VB'), ('up', 'RP'), ('the', 'DT'),
('words', 'NNS'), ('in', 'IN'), ('text', 'JJ'), ('format', 'NN'), ('for', 'IN'), ('a', 'D
T'), ('particular', 'JJ'), ('part', 'NN'), ('of', 'IN'), ('a', 'DT'), ('speech', 'NN'), (
'based', 'VBN'), ('on', 'IN'), ('its', 'PRP$'), ('definition', 'NN'), ('It', 'PRP'), ('is
', 'VBZ'), ('responsible', 'JJ'), ('for', 'IN'), ('text', 'JJ'), ('reading', 'NN'), ('in'
, 'IN'), ('a', 'DT'), ('language', 'NN'), ('It', 'PRP'), ('is', 'VBZ'), ('also', 'RB'), (
'called', 'VBN'), ('grammatical', 'JJ'), ('tagging', 'NN'), ('.', '.')]
```

```
#Code for Parsing
import nltk
```

```
from nltk import word_tokenize
from nltk.parse import RecursiveDescentParser
sentence = "the cat sat on the mat"
tokens = word_tokenize(sentence)
# Define a grammar for parsing the sentence
grammar = nltk.CFG.fromstring("""
            S -> NP VP
            NP -> DT NN
            VP -> V PP
            PP -> IN NP
            DT -> 'the'
            NN -> 'cat' | 'mat'
            V -> 'sat'
            IN -> 'on'
            """)
# Create a parser object
parser = RecursiveDescentParser(grammar)
print("\nAfter parsing: ")
for tree in parser.parse(tokens):
    print(tree)
```

Output

```
IDLE Shell 3.11.1
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec  6 2022, 19:58:39) [MSC v.1934 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
==================== RESTART: C:\Users\sumit\Desktop\Lab 9.py ====================
After parsing:
(S
  (NP (DT the) (NN cat))
  (VP (V sat) (PP (IN on) (NP (DT the) (NN mat)))))
```

Criteria	Total Marks	Marks Obtained	Comments
Concept(A)	2		
Implementation(B)	2		
Performance	2		
Total	6 (to be scaled down to 1)		

Lab-10

Aim: To implement 0/1 Knapsack using Brute Force approach

Theory:

A set of objects with different weights and their values is given. The objective of 0/1 knapsack problem is to find the most optimal subset of these objects such that the total weightof these objects should not exceed a certain amount and the summation of values of these objects should be the maximum possible value incurred.

Brute Force approach is a method in which all 2^n combinations of the objects are taken 1 by1 and the combination satisfying the above constraints is selected. E.g., if the objects with weights and values are given as follows and the maximum capacity is 40.

Item	Weight	Value
0	30	10
1	10	20
2	40	30
3	20	40

The most optimal solution is: Item1 and Item3 which account for a profit of 60 and a weightof 30.

CODE:

```
def knapsack_brute_force(values, weights, capacity):
  n = len(values)
  max_value = 0
  best_subset = None
# Generate all possible subsets of items
for i in range(2**n):
  subset = []
  subset_weight = 0
  subset_value = 0
  for j in range(n):
    if (i >> j) & 1==1:
      subset.append(j)
      subset_weight += weights[j]
      subset_value += values[j]
      # Check if the subset is feasible and has higher value than previous best
      if subset_weight <= capacity and subset_value > max_value:
        max_value = subset_value
        best_subset = subset
```

```
return max_value, best_subset
knapsack_brute_force([10,20,30,40],[30,10,40,20],40)
```

Output:

(60, [1, 3])

Criteria	Total Marks	Marks Obtained	Comments
Concept(A)	2		
Implementation(B)	2		
Performance	2		
Total	6 (to be scaled down to 1)		

Lab-11

Aim: To study about Fuzzy Scikit Libraries

THEORY:

Fuzzy logic is a branch of mathematics that deals with reasoning that is approximate ratherthan fixed and exact. It is used in various applications, such as control systems, decision making, and data analysis, to deal with uncertainty and imprecision.

Scikit-fuzzy is a fuzzy logic toolkit for Python that provides a comprehensive set of tools forfuzzy control, fuzzy reasoning, and fuzzy analysis. It is built on top of the SciPy and NumPylibraries and provides a user-friendly interface for working with fuzzy logic.

In this experiment, we will use the scikit-fuzzy library to implement a fuzzy logic system tocontrol the speed of a fan. We will use a simple set of rules to adjust the speed of the fan based on the temperature and humidity of the room.

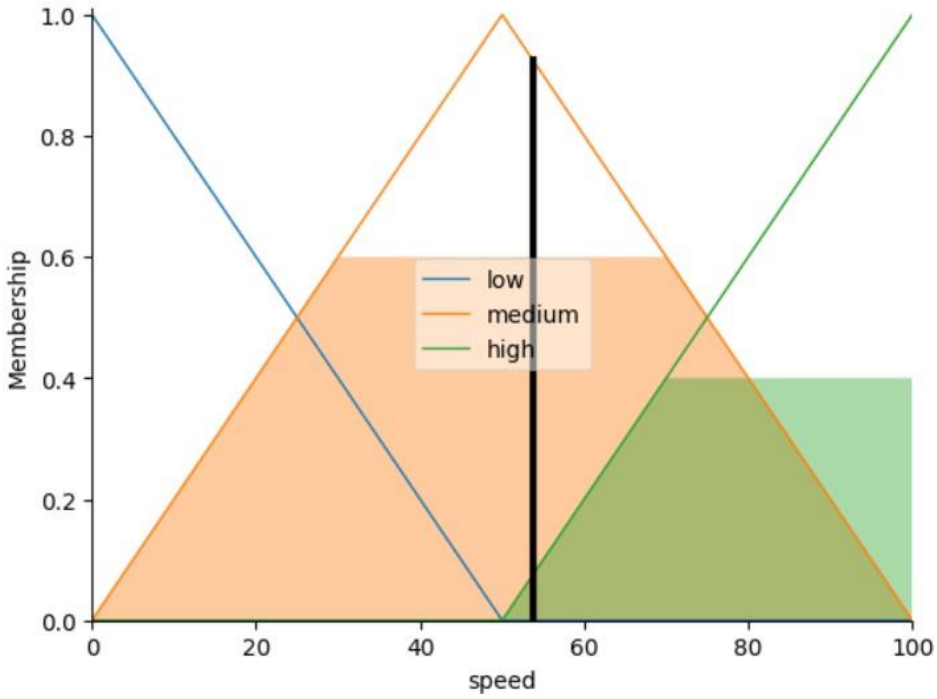
CODE:

```
#To get started, we need to install the scikit-fuzzy library. We can do this using pip:
!pip install scikit-fuzzy
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl

# Create input variables
temperature = ctrl.Antecedent(np.arange(0, 101, 1), 'temperature')
humidity = ctrl.Antecedent(np.arange(0, 101, 1), 'humidity')
# Create output variable
speed = ctrl.Consequent(np.arange(0, 101, 1), 'speed')

# Define fuzzy sets for input variables
temperature['cold'] = fuzz.trimf(temperature.universe, [0, 0, 50])
temperature['warm'] = fuzz.trimf(temperature.universe, [0, 50, 100])
temperature['hot'] = fuzz.trimf(temperature.universe, [50, 100, 100])
humidity['low'] = fuzz.trimf(humidity.universe, [0, 0, 50])
humidity['medium'] = fuzz.trimf(humidity.universe, [0, 50, 100])
humidity['high'] = fuzz.trimf(humidity.universe, [50, 100, 100])

# Define fuzzy sets for output variable
speed['low'] = fuzz.trimf(speed.universe, [0, 0, 50])
speed['medium'] = fuzz.trimf(speed.universe, [0, 50, 100])
speed['high'] = fuzz.trimf(speed.universe, [50, 100, 100])

# Define rules
rule1 = ctrl.Rule(temperature['cold'] & humidity['low'], speed['low'])
```

```
rule2 = ctrl.Rule(temperature['cold'] & humidity['medium'], speed['medium'])
rule3 = ctrl.Rule(temperature['cold'] & humidity['high'], speed['medium'])
rule4 = ctrl.Rule(temperature['warm'] & humidity['low'], speed['medium'])
rule5 = ctrl.Rule(temperature['warm'] & humidity['medium'], speed['medium'])
rule6 = ctrl.Rule(temperature['warm'] & humidity['high'], speed['high'])
rule7 = ctrl.Rule(temperature['hot'] & humidity['low'], speed['medium'])
rule8 = ctrl.Rule(temperature['hot'] & humidity['medium'], speed['high'])
rule9 = ctrl.Rule(temperature['hot'] & humidity['high'], speed['high'])
# Create control system
fan_ctrl = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5, rule6, rule7, rule8, rule9])
fan_speed = ctrl.ControlSystemSimulation(fan_ctrl)
# Set input values fan_speed.input['temperature'] = 70
fan_speed.input['humidity'] = 60
# Compute result
fan_speed.compute()
# Print result
print(fan_speed.output['speed'])
speed.view(sim=fan_speed)
```

Output:

```
53.76811594202898
```



Criteria	Total Marks	Marks Obtained	Comments
Concept(A)	2		
Implementation(B)	2		
Performance	2		
Total	6 (to be scaled down to 1)		

Criteria	Total Marks	Marks Obtained	Comments
Concept(A)	2		
Implementation(B)	2		
Performance	2		
Total	6 (to be scaled down to 1)		

OPEN – ENDED EXPERIMENT

Aim: Insurance Claims Fraud detection Using machine learning algorithms.

Theory:

Fraud in insurance claims refers to the act of intentionally deceiving an insurer to obtain money or benefits to which one is not entitled. It can involve fabricating or exaggerating claims, hiding pre-existing conditions, or even staging accidents or injuries. Fraud detection in insurance claims is a crucial task as it helps prevent losses and maintain the financial stability of insurance companies.

Using machine learning algorithms for fraud detection in insurance claims can help automate the process of identifying fraudulent claims. However, implementing such a system comes with ethical considerations that must be considered. The algorithms used to detect fraud may unfairly target certain groups, and privacy concerns may arise as personal information is collected and analysed.

One of the primary challenges of implementing a fraud detection system for insurance claims is dealing with the class imbalance problem. The vast majority of insurance claims are legitimate, while only a small fraction are fraudulent. This imbalance makes it challenging to train machine learning models to accurately detect fraudulent claims without overfitting to the majority class.

To address this issue, machine learning algorithms can be trained on data sets that include both legitimate and fraudulent claims. This can help the algorithm learn to identify patterns and features that distinguish between the two. However, this approach requires careful consideration of privacy concerns and potential biases in the data set.

Ultimately, the goal of using machine learning for fraud detection in insurance claims should be to balance the need for accurate fraud detection with ethical considerations and privacy concerns.

Code:

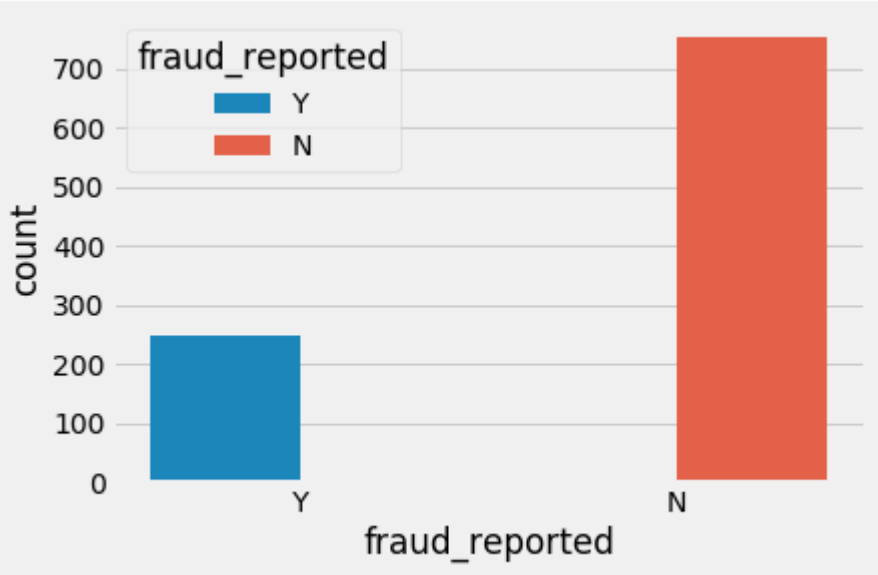
```
# import all libraries

import pandas as pd
from matplotlib import pyplot as plt
import seaborn as sns
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
import numpy as np
import sklearn.metrics
from pylab import rcParams
%matplotlib inline
pd.set_option('display.max_columns', 500)
```

```
pd.set_option('display.max_rows', 500)
```

```
#load & view raw data

df = pd.read_csv('insurance_claims.csv')
df.head(10)
```

	months_as_customer	age	policy_number	policy_bind_date	policy_state	policy_csl	policy_deductable	policy_annual_premium	umbrella_limit	insured_zip	insured_sex
0	328	48	521585	2014-10-17	OH	250/500	1000	1406.91	0	466132	MALE
1	228	42	342868	2006-06-27	IN	250/500	2000	1197.22	5000000	468176	MALE
2	134	29	687698	2000-09-06	OH	100/300	2000	1413.14	5000000	430632	FEMALE
3	256	41	227811	1990-05-25	IL	250/500	2000	1415.74	6000000	608117	FEMALE
4	228	44	367455	2014-06-06	IL	500/1000	1000	1583.91	6000000	610706	MALE
5	256	39	104594	2006-10-12	OH	250/500	1000	1351.10	0	478456	FEMALE
6	137	34	413978	2000-06-04	IN	250/500	1000	1333.35	0	441716	MALE
7	165	37	429027	1990-02-03	IL	100/300	1000	1137.03	0	603195	MALE
8	27	33	485665	1997-02-05	IL	100/300	500	1442.99	0	601734	FEMALE
9	212	42	636550	2011-07-25	IL	100/300	500	1315.68	0	600983	MALE

```
# Print the shape and description of the data

df.shape
df.nunique()

#Check for the Fraud cases

plt.style.use('fivethirtyeight')
ax = sns.countplot(x='fraud_reported', data=df, hue='fraud_reported')
```



df['fraud_reported'].value_counts()

```
Out[8]: N    753
        Y    247
        Name: fraud_reported, dtype: int64
```

 # here we see that almost 25% of fraud is reported. Let's try to look for an indicative variable. Let's analyze location. This dataset only has information from the mid-Atlantic states from the USA.

df['incident_state'].value_counts()

```
Out[9]: NY    262
        SC    248
        WV    217
        VA    110
        NC    110
        PA     30
        OH     23
        Name: incident_state, dtype: int64
```

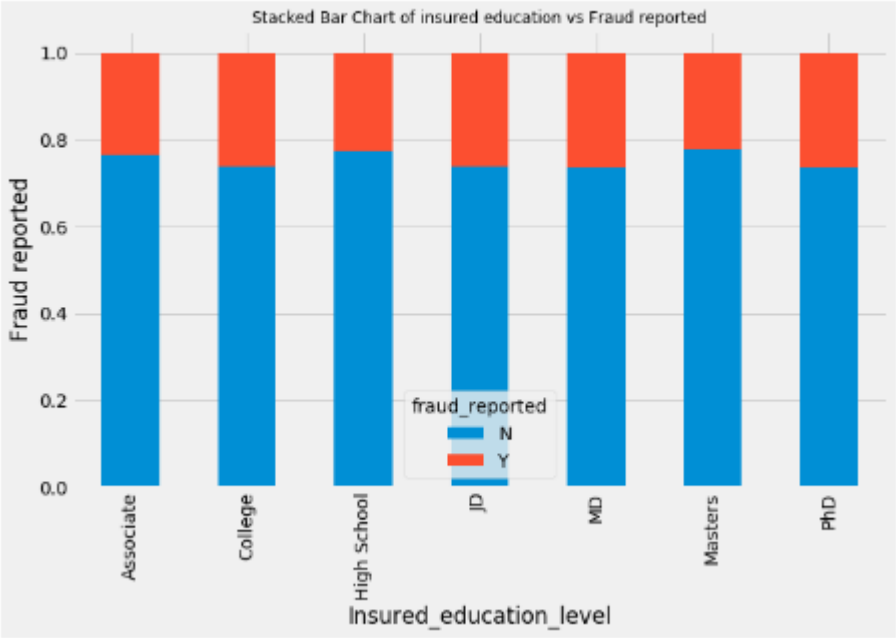
we will consider the conditions like checking age between 19 – 23 are not and checking in which months the frauds happened more

```
plt.rcParams['figure.figsize'] = [15, 8]
ax= plt.style.use('fivethirtyeight')
table=pd.crosstab(df.age, df.fraud_reported)
table.div(table.sum(1).astype(float), axis=0).plot(kind='bar', stacked=True)
plt.title('Stacked Bar Chart of Age vs Fraud Reported', fontsize=12)
plt.xlabel('Age')
plt.ylabel('Fraud reported')
plt.show()
```

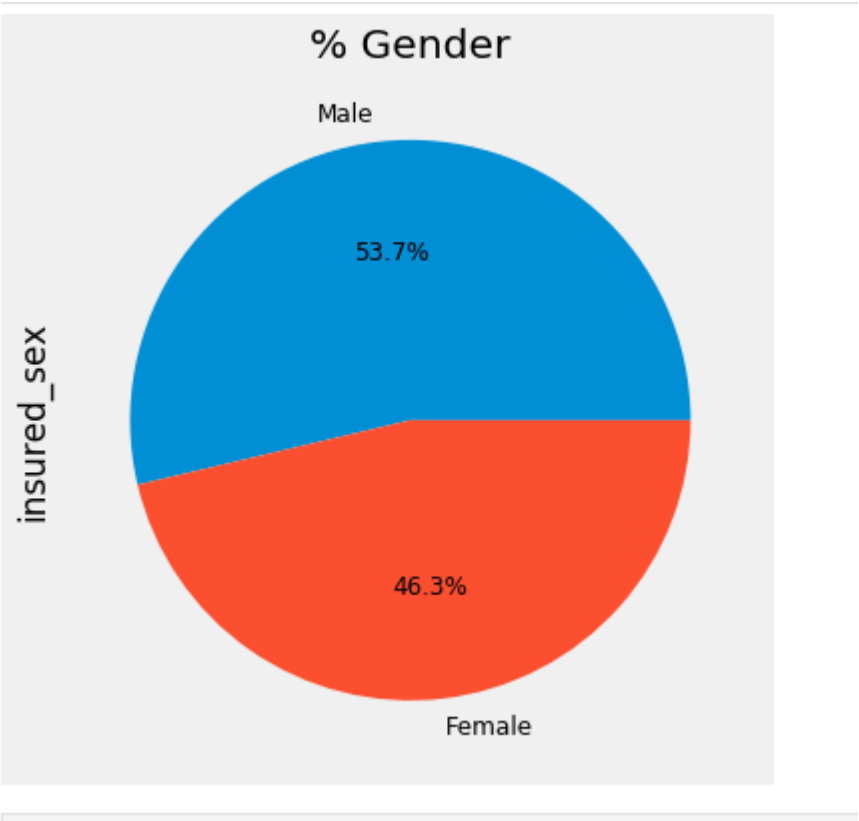
```
plt.style.use('fivethirtyeight')
fig = plt.figure(figsize=(18,6))
ax = df.groupby('incident_date').total_claim_amount.count().plot.bar(ylim=0)
ax.set_ylabel('Claim amount ($)')
plt.show()
```

```
plt.rcParams['figure.figsize'] = [10, 6]
table=pd.crosstab(df.insured_education_level, df.fraud_reported)
table.div(table.sum(1).astype(float), axis=0).plot(kind='bar', stacked=True)
plt.title('Stacked Bar Chart of insured education vs Fraud reported', fontsize=12)
plt.xlabel('Insured_education_level')
plt.ylabel('Fraud reported')
```

```
Text(0, 0.5, 'Fraud reported')
```

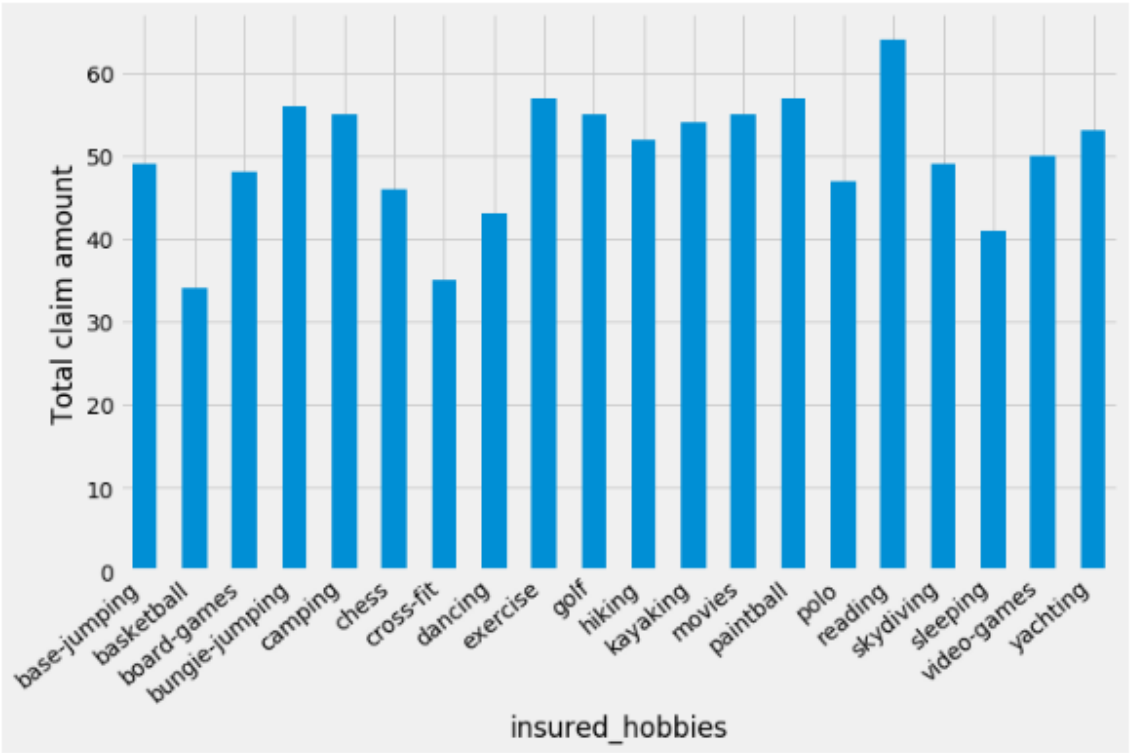



```
ax = (df['insured_sex'].value_counts()*100.0 /len(df))\
.plot.pie(autopct='%.1f%%', labels = ['Male', 'Female'], fontsize=12)
ax.set_title('% Gender')
plt.show()
```



 # Like this we will check all the factors

```
plt.style.use('fivethirtyeight')
fig = plt.figure(figsize=(10,6))
ax= df.groupby('insured_hobbies').total_claim_amount.count().plot.bar(ylim=0)
ax.set_ylabel('Total claim amount')
ax.set_xticklabels(ax.get_xticklabels(), rotation=40, ha="right")
plt.show()
```



 # next we will do data preprocessing

df['fraud_reported'].replace(to_replace='Y', value=1, inplace=**True**)
df['fraud_reported'].replace(to_replace='N', value=0, inplace=**True**)

df.head()

Out[31]:

	months_as_customer	age	policy_number	policy_bind_date	policy_state	policy_csl	policy_ded
0	328	48	521585	2014-10-17	OH	250/500	
1	228	42	342868	2006-06-27	IN	250/500	
2	134	29	687698	2000-09-06	OH	100/300	
3	256	41	227811	1990-05-25	IL	250/500	
4	228	44	367455	2014-06-06	IL	500/1000	

df.describe(include='all')

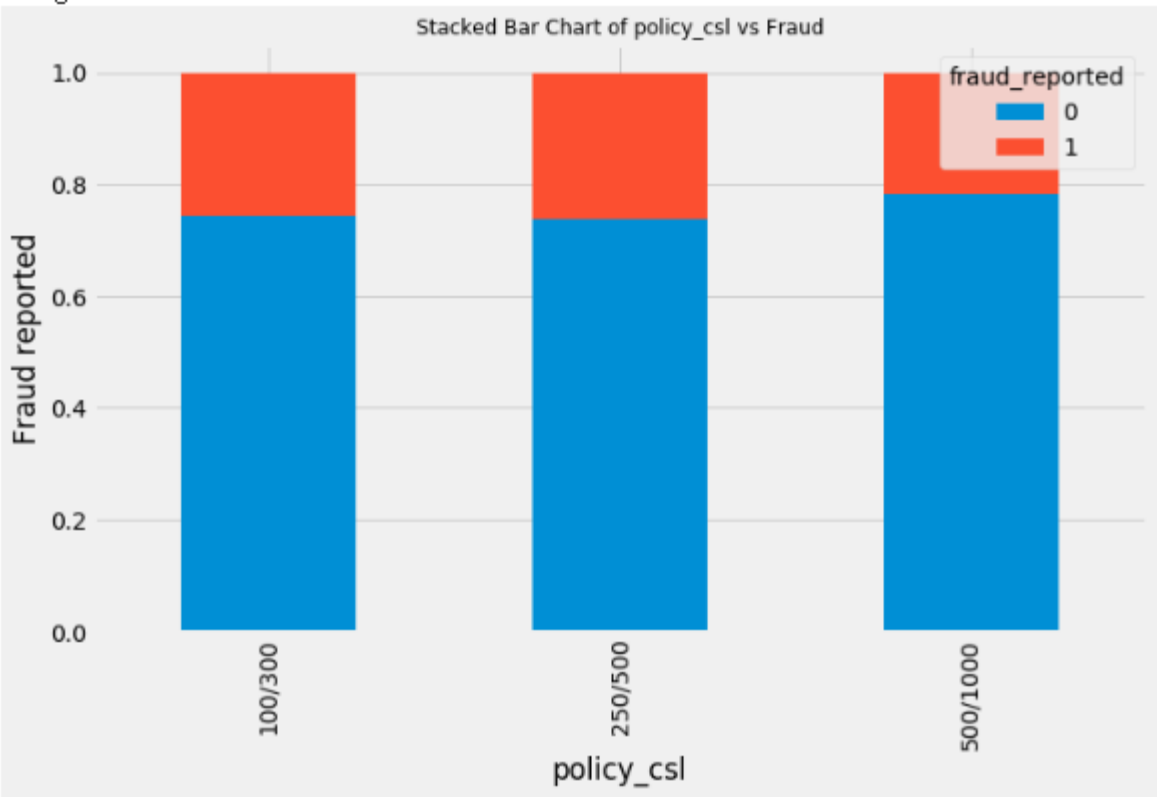
Out[34]:

	months_as_customer	age	policy_number	policy_bind_date	policy_state	policy_csl
count	1000.000000	1000.000000	1000.000000	1000	1000	1000
unique	NaN	NaN	NaN	951	3	3
top	NaN	NaN	NaN	1992-08-05	OH	250/500
freq	NaN	NaN	NaN	3	352	351
mean	203.954000	38.948000	546238.648000	NaN	NaN	NaN
std	115.113174	9.140287	257063.005276	NaN	NaN	NaN
min	0.000000	19.000000	100804.000000	NaN	NaN	NaN
25%	115.750000	32.000000	335980.250000	NaN	NaN	NaN
50%	199.500000	38.000000	533135.000000	NaN	NaN	NaN
75%	276.250000	44.000000	759099.750000	NaN	NaN	NaN
max	479.000000	64.000000	999435.000000	NaN	NaN	NaN

 # Here we see some NaN values in the dataset this means the missing values then we will deal with this

```
plt.style.use('fivethirtyeight')
fig = plt.figure(figsize=(10,6))
table=pd.crosstab(df.policy_csl, df.fraud_reported)
table.div(table.sum(1).astype(float), axis=0).plot(kind='bar', stacked=True)
plt.title('Stacked Bar Chart of policy_csl vs Fraud', fontsize=12)
plt.xlabel('policy_csl')
plt.ylabel('Fraud reported')
plt.show()
```

```
<Figure size 720x432 with 0 Axes>
```



policy_csl looks like an unavoidable predictor

df.select_dtypes(include=['object']).columns *# checking categorcial columns*

```
[42]: Index(['policy_bind_date', 'policy_state', 'policy_csl', 'insured_zip',
       'insured_sex', 'insured_education_level', 'insured_occupation',
       'insured_hobbies', 'insured_relationship', 'incident_date',
       'incident_type', 'collision_type', 'incident_severity',
       'authorities_contacted', 'incident_state', 'incident_city',
       'incident_location', 'property_damage', 'police_report_available',
       'auto_make', 'auto_model', 'csl_per_person', 'csl_per_accident',
       'incident_period_of_day'],
      dtype='object')
```

 # dropping unimportant columns

```
df = df.drop(columns = [
    'policy_number',
    'policy_csl',
    'insured_zip',
    'policy_bind_date',
    'incident_date',
    'incident_location',
    '_c39',
```

```
    'auto_year',
    'incident_hour_of_the_day'])
```

```
df.head(2)
```

ut[43]:

	months_as_customer	age	policy_state	policy_deductable	policy_annual_premium	umbrella_lir
0	328	48	OH	1000	1406.91	
1	228	42	IN	2000	1197.22	5000(

 # we will do label encoding

```
from sklearn.preprocessing import LabelEncoder
X['collision_en'] = LabelEncoder().fit_transform(dummies['collision_type'])
X[['collision_type', 'collision_en']]
```

Out[59]:

	collision_type	collision_en
0	Side Collision	3
1	?	0
2	Rear Collision	2
3	Front Collision	1
4	?	0
5	Rear Collision	2
6	Front Collision	1
7	Front Collision	1
8	Front Collision	1
9	Rear Collision	2
10	Front Collision	1
11	Front Collision	1
12	Rear Collision	2
13	?	0
14	Rear Collision	2
15	Side Collision	3
16	Rear Collision	2
17	Side Collision	3
18	Side Collision	3
19	Side Collision	3

 # We now have a dataset that we could use to evaluate an algorithm sensitive to missing values like LDA.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score

# evaluate an LDA model on the dataset using k-fold cross validation
model = LinearDiscriminantAnalysis()
kfold = KFold(n_splits=5, random_state=7)
result = cross_val_score(model, X, y, cv=kfold, scoring='accuracy')
print(result.mean())
```

```
0.841
C:\Users\Sarit\Anaconda3\envs\python\lib\site-packages\sklearn\discriminant_ana
sis.py:388: UserWarning: Variables are collinear.
  warnings.warn("Variables are collinear.")
```

```
[66]: print("Accuracy: %0.2f (+/- %0.2f)" % (result.mean(), result.std() * 2))

Accuracy: 0.84 (+/- 0.07)
```

Training and Testing

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.8, random_state=7)
print('length of X_train and X_test: ', len(X_train), len(X_test))
print('length of y_train and y_test: ', len(y_train), len(y_test))
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, recall_score, classification_report,
cohen_kappa_score
from sklearn import metrics

# Baseline Random forest based Model
rfc = RandomForestClassifier(n_estimators=200)

kfold = KFold(n_splits=5, random_state=7)
result2 = cross_val_score(rfc, X_train, y_train, cv=kfold, scoring='accuracy')
print(result2.mean())
```

Standardizing the data and recheck the data distribution.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler(with_mean=False)
X_train_scaled = scaler.fit_transform(X_train)
```

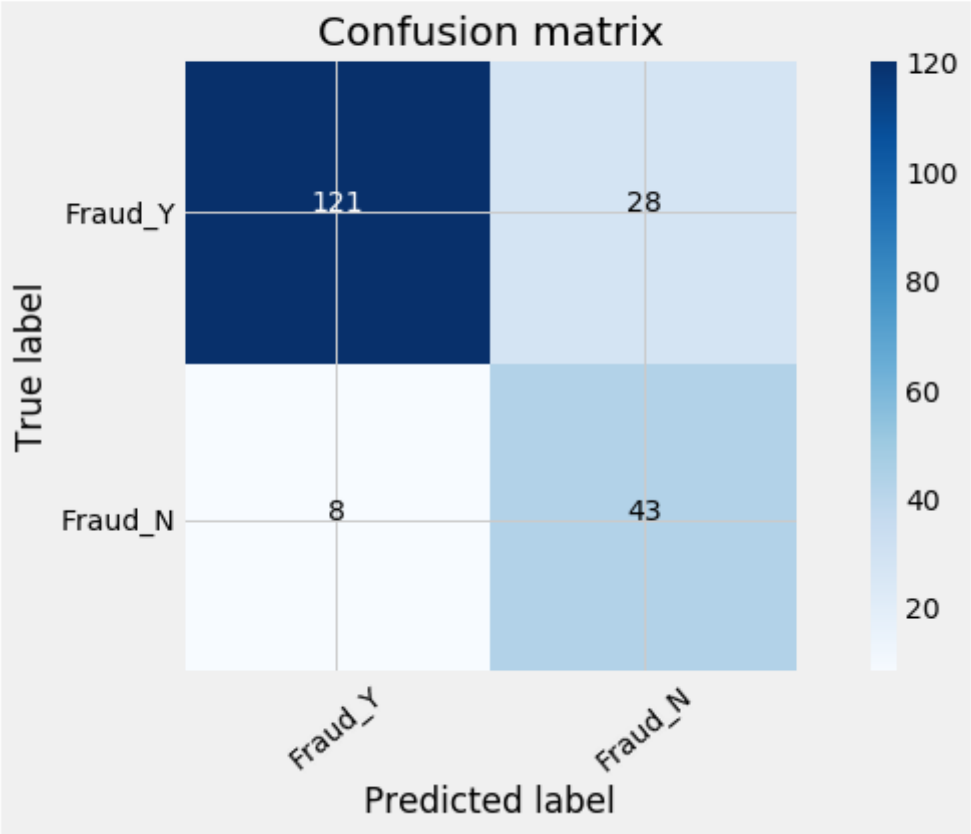
```
X_test_scaled = scaler.transform(X_test)
X_train_scaled
```

```
array([[0.        , 0.        , 2.08816082, ..., 0.90375361, 1.84093195,
        1.81430422],
       [0.        , 0.        , 2.08816082, ..., 2.10117545, 1.60502305,
        2.14417772],
       [0.        , 2.16950399, 0.        , ..., 2.69678424, 2.74665362,
        0.82468374],
       ...,
       [2.11480423, 0.        , 0.        , ..., 1.26980485, 2.58657258,
        3.7935452 ],
       [0.        , 2.16950399, 0.        , ..., 0.10960856, 0.22327092,
        2.80392471],
       [0.        , 2.16950399, 0.        , ..., 3.29239303, 2.51495738,
        1.15455723]])
```

```
#Confusion matrix

from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score

# calculate AUC
auc = roc_auc_score(y_test, rgs_pred)
print('AUC: %.3f' % auc)

# calculate roc curve
fpr, tpr, thresholds = roc_curve(y_test, rgs_pred)

# plot no skill
plt.rcParams['figure.figsize'] = [10, 6]
plt.plot([0, 1], [0, 1], linestyle='--')

# plot the roc curve for the model
plt.plot(fpr, tpr, marker='.')


from sklearn.metrics import confusion_matrix
import itertools

#Evaluation of Model - Confusion Matrix Plot
def plot_confusion_matrix(cm, classes, title ='Confusion matrix', normalize = False, cmap =
plt.cm.Blues):

    print('Confusion matrix')

    print(cm)

    plt.style.use('fivethirtyeight')
    fig = plt.figure(figsize=(10,6))
```

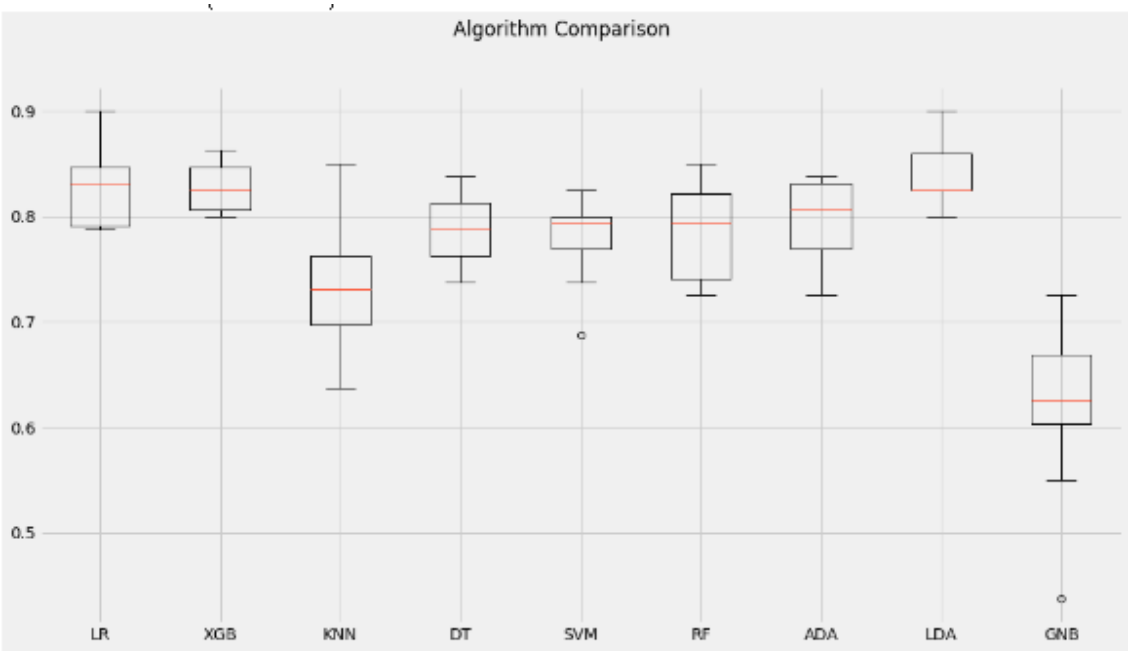
```
plt.imshow(cm, interpolation='nearest', cmap=cmap)
plt.title(title)
plt.colorbar()
tick_marks = np.arange(len(classes))
plt.xticks(tick_marks, classes, rotation=40)
plt.yticks(tick_marks, classes)

fmt = '.2f' if normalize else 'd'
thresh = cm.max() / 2.
for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
    plt.text(j, i, format(cm[i, j], fmt),
             horizontalalignment="center",
             color="white" if cm[i, j] > thresh else "black")

plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.tight_layout()
```

```
Confusion matrix
[[121  28]
 [  8  43]]
<Figure size 720x432 with 0 Axes>
```



 # comparison of all algorithms

```
from sklearn.model_selection import KFold, cross_val_score
from xgboost import XGBClassifier
from sklearn.linear_model import LogisticRegressionCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

xgb = XGBClassifier()
logreg= LogisticRegressionCV(solver='lbfgs', cv=10)
knn = KNeighborsClassifier(5)
svcl = SVC()
adb = AdaBoostClassifier()
dt = DecisionTreeClassifier(max_depth=5)
rf = RandomForestClassifier()
lda = LinearDiscriminantAnalysis()
gnb = GaussianNB()

# prepare configuration for cross validation test harness
seed = 7
# prepare models
models = []
models.append(('LR', LogisticRegressionCV(solver='lbfgs', max_iter=5000, cv=10)))
models.append(('XGB', XGBClassifier()))
models.append(('KNN', KNeighborsClassifier()))
models.append(('DT', DecisionTreeClassifier()))
models.append(('SVM', SVC(gamma='auto')))
models.append(('RF', RandomForestClassifier(n_estimators=200)))
models.append(('ADA', AdaBoostClassifier(n_estimators=200)))
models.append(('LDA', LinearDiscriminantAnalysis()))
models.append(('GNB', GaussianNB()))

# evaluate each model in turn
results = []
names = []
scoring = 'accuracy'
for name, model in models:
    kfold = model_selection.KFold(n_splits=10, random_state=seed)
    cv_results = model_selection.cross_val_score(model, x_train_scaled, y_train, cv=kfold,
scoring=scoring)
    results.append(cv_results)
    names.append(name)
    msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
    print(msg)

# boxplot algorithm comparison
plt.rcParams['figure.figsize'] = [15, 8]
fig = plt.figure()
fig.suptitle('Algorithm Comparison')
ax = fig.add_subplot(111)
plt.boxplot(results)
```

```
ax.set_xticklabels(names)
plt.show()
```

```
LR: 0.826250 (0.034664)
XGB: 0.828750 (0.023083)
KNN: 0.735000 (0.055283)
DT: 0.787500 (0.032596)
SVM: 0.780000 (0.038810)
RF: 0.785000 (0.045689)
ADA: 0.797500 (0.036142)
LDA: 0.840000 (0.029475)
```